CHARUSAT
CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

PROGRAMMING WITH C++

UNIT 1

OVERVIEW OF C++ PROGRAMMING

DEPSTAR

Devang Patel Institute of Advance Technology and Research

Outlines

- Introduction to C++ Programming Language
  - Brief history of C++
  - Key features of C++
  - Role of C++ in modern software development
- Fundamental Types and Portability
  - Understanding fundamental data types in C++
  - Challenges related to data type portability
- Introduction to Object-Oriented Programming (OOP)
  - Core principles of OOP
  - Differences between Object-Oriented and Procedural programming,
  - The significance of OOP in software development
- Standard Library and Namespaces
  - Introduction to the C++ Standard Library
  - Overview of namespaces in C++
  - structure of C++ program

Brief history of C++

- C++ was developed by **Bjarne Stroustrup** at Bell Labs in the early 1980s.
- Initially, it was designed as an enhancement to the C programming language, adding object-oriented features while maintaining C's efficiency and low-level capabilities.
- The first version of C++ was called "C with Classes," and its primary goal was to provide a more effective way to manage large-scale software projects.

Cont.

- **C++98:** The first standardized version of C++ by ISO (International Organization for Standardization).
- **C++03:** A bug-fix update to C++98.
- **C++11:** Introduced significant features like auto keyword, nullptr, lambda expressions, and range-based for loops.
- **C++14 and C++17:** Introduced further language enhancements and optimizations.
- **C++20:** Added features like concepts, coroutines, and ranges, along with continued optimizations and tools for modern software development.
- **C++23:** Refined features from C++20 and introduced further improvements in performance and ease of use.

Key Features of C++

- **Efficiency**: C++ allows direct control over system resources, such as memory and hardware, making it one of the most efficient languages for high-performance applications.
- **Performance**: C++ generates optimized machine code, which makes it one of the fastest programming languages, suitable for performance-critical applications like real-time systems and games.
- **Multi-Paradigm Support**: C++ supports procedural, object-oriented, and generic programming, allowing developers to choose the best paradigm for the task at hand.
- **Object-Oriented Programming (OOP):** C++ supports key OOP concepts such as classes, inheritance, polymorphism, and encapsulation, which help in organizing complex software systems and improving code reuse.
- **Templates and Generic Programming**: C++ provides templates, which enable writing generic and reusable code. This allows the creation of functions and classes that work with any data type.
- **Standard Template Library (STL):** The STL is a powerful library that provides pre-built templates for common data structures (e.g., vectors, lists, maps) and algorithms (e.g., sorting, searching), boosting productivity and efficiency.
- **Low-Level Memory Management**: C++ gives programmers direct control over memory allocation and deallocation using pointers and manual memory management, allowing precise control over system resources.

Key Features of C++

- **Function Overloading and Operator Overloading:** C++ supports function overloading, where multiple functions with the same name can exist but with different parameters, and operator overloading, allowing operators (e.g., +, *) to be customized for user-defined types.
- **Exception Handling:** C++ provides robust exception handling mechanisms (using try, catch, and throw), allowing developers to manage runtime errors and maintain program stability.
- **RAII (Resource Acquisition Is Initialization):** C++ uses RAII to manage resources like memory, file handles, and locks automatically. Objects acquire resources when they are created and release them when they go out of scope.
- **Const Keyword**: The const keyword ensures that variables or data members cannot be modified, which improves program safety and optimizes performance by enabling compiler optimizations.
- **Namespaces:** Namespaces allow the grouping of functions, classes, and variables into logical scopes, avoiding name conflicts in large projects.
- **Smart Pointers:** C++ supports smart pointers (e.g., std::unique_ptr, std::shared_ptr) to manage memory automatically and help prevent common issues like memory leaks and dangling pointers.
- **Lambda Expressions:** C++11 introduced lambdas, which allow the definition of anonymous functions (or function objects) in place, making it easier to write short, inline functions.
- **Move Semantics and Rvalue References:** Introduced in C++11, move semantics allow efficient transfer of resources (like memory or ownership) from one object to another, reducing unnecessary copying and improving performance.

Role of C++ in modern software development

- **Systems Programming:** C++ is used for developing operating systems, device drivers, and embedded systems due to its low-level hardware control and performance.
- **Game Development:** It's the primary language for game engines (e.g., Unreal Engine), offering real-time rendering and physics simulation with high performance.
- **High-Performance Computing:** C++ is crucial for scientific simulations, financial modeling, and supercomputing, where speed and resource management are critical.
- **Real-Time Systems:** Used in industries like automotive, aerospace, and robotics, C++ helps build systems with strict timing and resource constraints.
- **Machine Learning & AI:** While Python is popular for AI, C++ powers high-performance libraries (e.g., TensorFlow) for fast computation and low-latency applications.
- **Database Systems:** C++ is used to build high-performance database engines (e.g., MySQL, PostgreSQL) for efficient data handling.
- **Multimedia & Graphics:** C++ is foundational in graphics, video processing, and other multimedia applications requiring intensive processing.
- **Cross-Platform Development:** Frameworks like Qt allow C++ applications to run across different platforms with high performance.

C++ DATA TYPES

- C++ data types divided mainly in three categories as follows:

  - Basic data types
  - User defined data types
  - Derived data types

BASIC DATA TYPES

TYPES	BYTES	RANGE
char	1	-128 to 127
unsigned char	1	0 to 255
signed char	1	-128 to 127
int	2	-32768 to 32767
unsigned int	2	0 to 65535
signed int	2	-32768 to 32767
short int	2	-32768 to 32767
unsigned short int	2	0 to 65535
signed short int	2	-32768 to 32767
long int	4	-2147483648 to 2147483647
signed long int	4	-2147483648 to 2147483647
unsigned long int	4	0 to 4294967295
float	8	3.4E-38 to 3.4E_38
double	10	1.7E-308 to 1.7E+308
long double		3.4E-4932 to 1.1E+4932

Example

```
#include <iostream>
using namespace std;

int main() {
    int age = 30;                 // Integer initialization
    float height = 5.9;           // Float initialization
    double pi = 3.14159;          // Double initialization
    char grade = 'A';             // Char initialization
    bool isStudent = true;        // Boolean initialization

    return 0;
}
```

USER-DEFINED DATA TYPES

There are following user defined data types:

- Struct
- Union
- Enum
- class

Struct

- A structure is a collection of simple variable
- The variable in a structure can be of different type such as int,char,float,char,etc.
- This is unlike the array which has all variables of same data type
- The data items in a structure is called the member of structure.

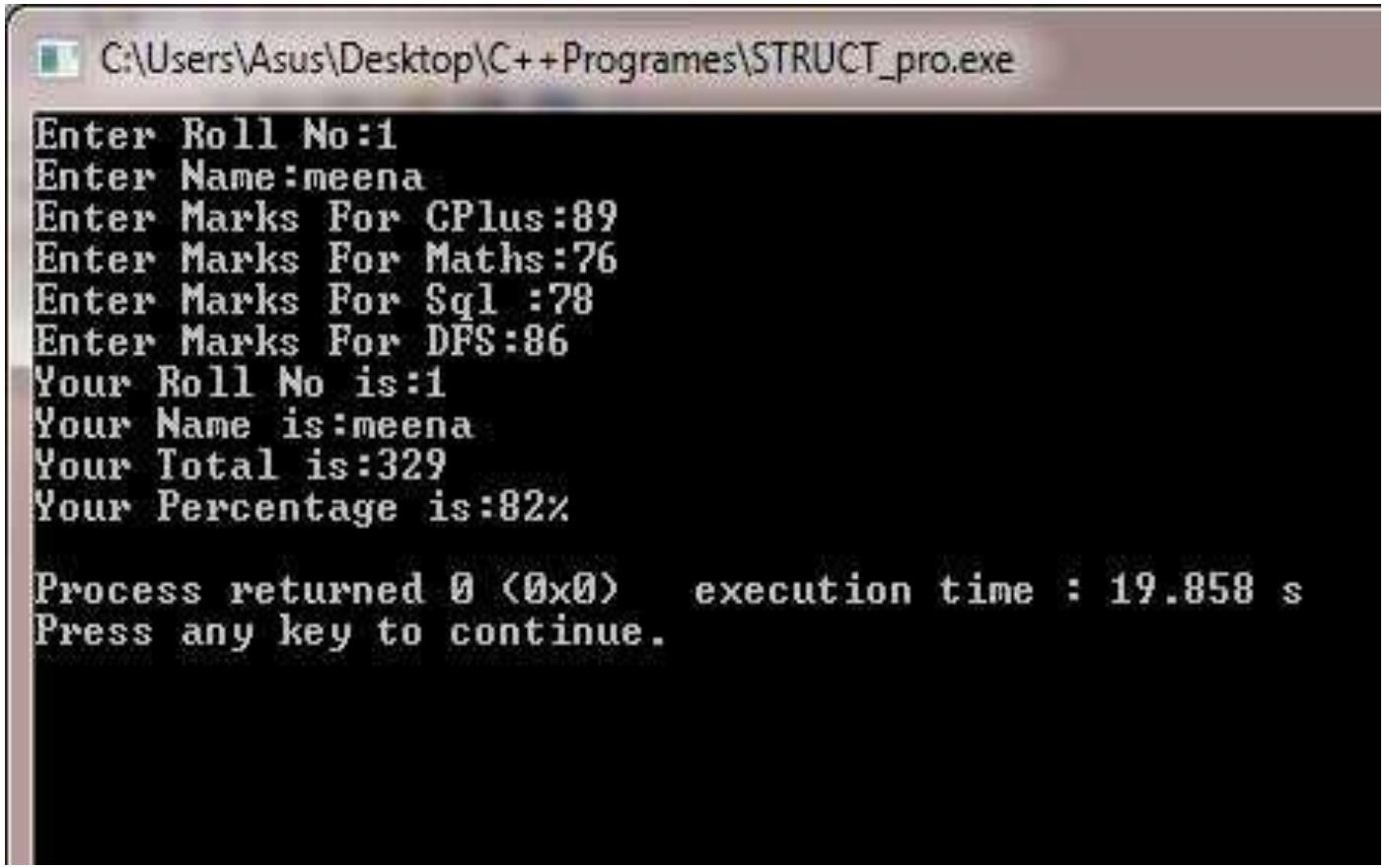
Syntax:

```
struct structure_name{
        declaration of data member;
};
```

- You can access data member of stucture by using only structure variable with dot operator.

PROGRAMME OF STRUCTURE

```
#include<iostream>
using namespace std;

struct student{
        int Roll_No;
        char name[15];
        int cPlus,maths,sql,dfs,total;
        float per;
};

int main(){
        struct student s;
        cout<<"Enter Roll No:";
        cin>>s.Roll_No;

        cout<<"Enter Name:";
        cin>>s.name;

        cout<<"Enter Marks For CPlus:";
        cin>>s.cPlus;

        cout<<"Enter Marks For Maths:";
        cin>>s.maths;

        cout<<"Enter Marks For Sql :";
        cin>>s.sql;

        cout<<"Enter Marks For DFS:";
        cin>>s.dfs;

        s.total=s.cPlus+s.maths+s.sql+s.dfs;
        s.per=s.total/4;

        cout<<"Your Roll No is:"<<s.Roll_No<<"\n";
        cout<<"Your Name is:"<<s.name<<"\n";
        cout<<"Your Total is:"<<s.total<<"\n";
        cout<<"Your Percentage is:"<<s.per<<"%"<<"\n";
return 0;
}
```


C:\Users\Asus\Desktop\C++Programes\STRUCT_pro.exe
Enter Roll No:1
Enter Name:meena
Enter Marks For CPlus:89
Enter Marks For Maths:76
Enter Marks For Sql :78
Enter Marks For DFS:86
Your Roll No is:1
Your Name is:meena
Your Total is:329
Your Percentage is:82%
Process returned 0 (0x0) execution time : 19.858 s
Press any key to continue.

UNION

- Union is also like a structure means union is also used to storing different data types ,but the difference between structure and union is that structure consume the memory of addition of all elements of different data types but a union consume the memory that is highest among all elements.

- It consume memory of highest variable and it will share the data among all other variable.

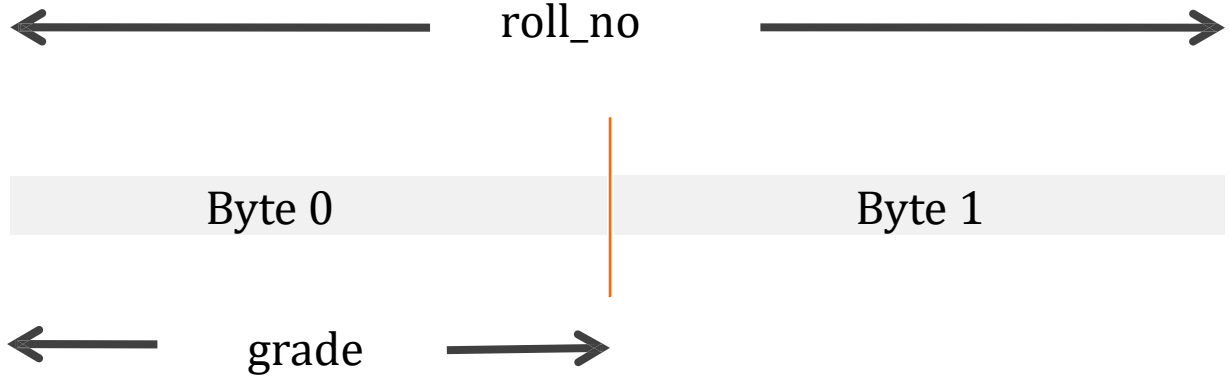
- For example: If a union contains variable like int, char,float then the memory will be consume of float variable because float is highest among all variable of data type.

Syntax:

```
union union_name              //union declaration
{
    data_type member1;
    data_type member2;
};
```

Example ,

```
union stu{              //occupies 2 bytes
    int roll_no;        //occupies 1 bytes
    char grade ;
};
```



Memory representation in union

Struct

- Define with '**struct**' keyword.
- All member can be manipulated simultaneosly.
- The size of object is equal to the sum of individual size of the member object.
- Members are allocated distinct memory location.

UNION

- Define with '**union**' keyword.
- Member can be manipulated one at a time.
- The size of object is equal to the size of largest member object.
- Members are share common memory space.

ENUM IN C++

- An enumeration type is user defined type that enable the user to define the range of values for the type.

  Syntax:

  ```
  enum [enum-type] {
      enum-list
  };
  ```

- Name constants are used to represent the value of an enumeration for example:

  ```
  enum char{a,b,c};
  ```

- The default value assigned to the enumeration constant are zero-based so in above example a=0,b=1 and so on.

ENUM IN C++

- The user also can assign value to one or more enumeration constant,and subsequent values that are not assigned will be incremented.

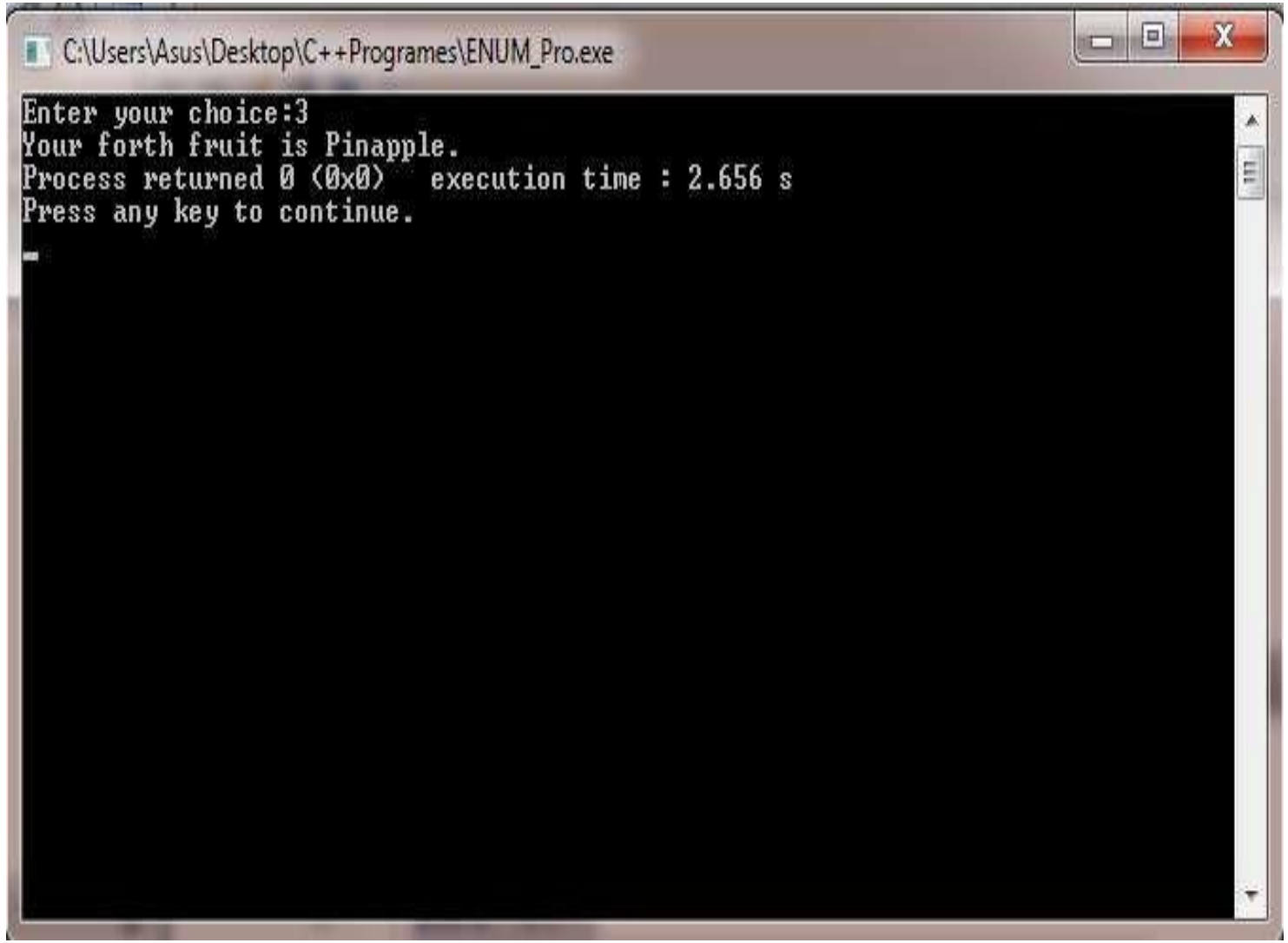
For example :

enum char{a=3,b=7,c,d};

here, value of c is 8 and d is 9.

Example of Enum

```
#include<iostream>
using namespace std;

int main()
{
        enum Fruits{
          apple,orange,guava,pinapple
        };

        Fruits myfruit;
        int i;
        cout<<"Enter your choice:";
        cin>>i;
        switch(i)
        {
        case apple:
        cout<<"Your first fruit is Apple.";  break;
        case orange:
        cout<<"Your second fruit is Orange.";  break;
        case guava:
        cout<<"Your third fruit is Guava.";
        break;
        case pineapples:
        cout<<"Your forth fruit is Pineapples.";  break;

        }
return 0;
}
```


C:\Users\Asus\Desktop\C++Programes\ENUM_Pro.exe
Enter your choice:3
Your forth fruit is Pinapple.
Process returned 0 (0x0)   execution time : 2.656 s
Press any key to continue.

CLASS

- The class type enables us to create sophisticated use defined type.
- We provide data items for the class and the operation that can be performed on the data.

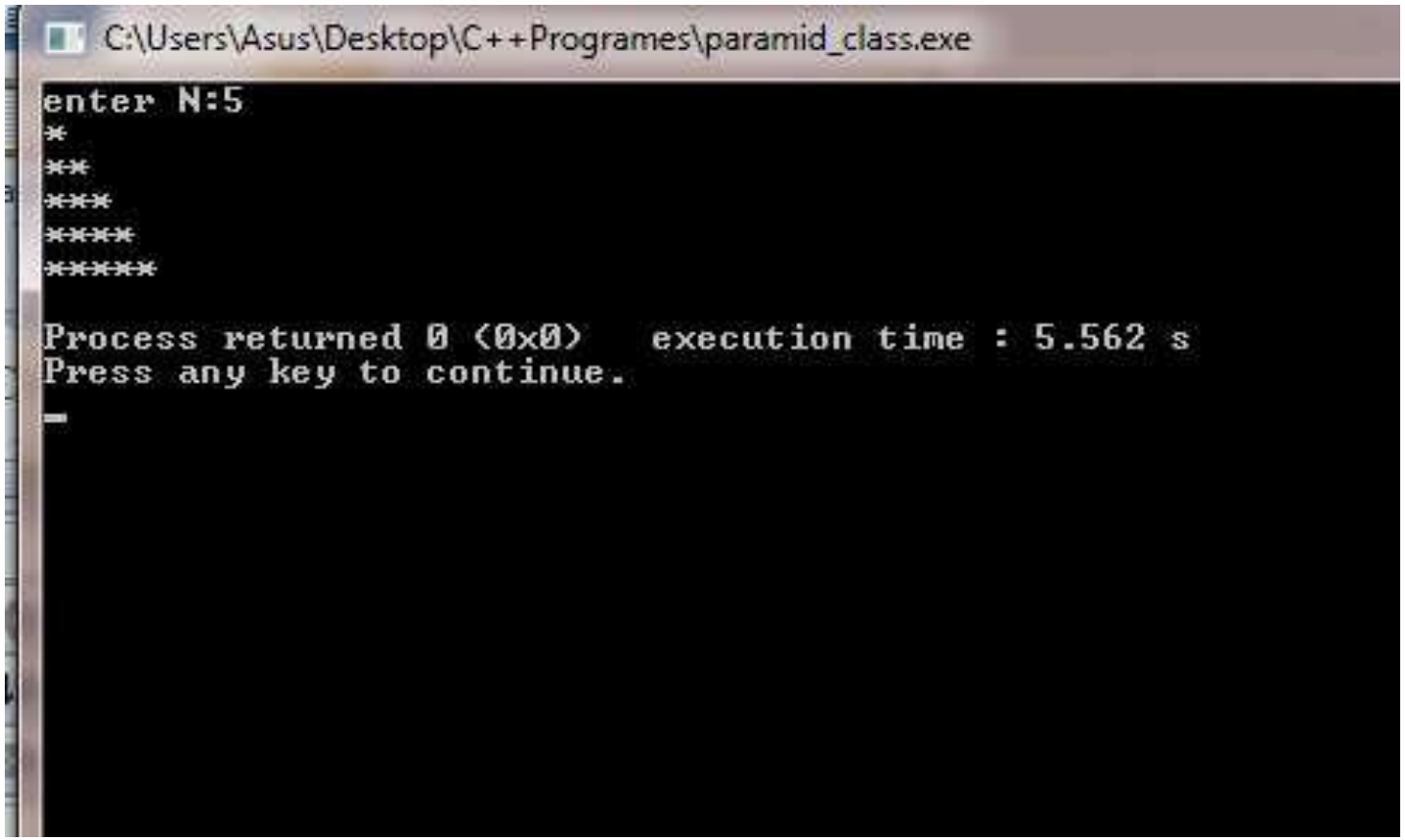
Syntax:

```
class class_name{
    data member variables;
    data member methods/functions;
};
```

Class Example

```
#include<iostream>
using namespace std;

class pyramid
{
      int i,n,j;

  public:
       void getdata();
       void buildpyramid();
};

void pyramid::getdata()
{
              cout<<"enter N:";
            cin>>n;
}
```

```
void piramid::buildpyramid()
{
        for(i=1;i<=n;i++)
        {
                for(j=1;j<=i;j++)
                {
                                cout<<"*";
                }
        cout<<"\n";
        }
}
int main()
{
                pyramid p;
                p.getdata();
                p.buildpyramid();
                return 0;
}
```


C:\Users\Asus\Desktop\C++Programes\paramid_class.exe
enter N:5
*
**
***
****
*****
Process returned 0 (0x0)   execution time : 5.562 s
Press any key to continue.

Derived Data Types

- The derived data type are:
  - array
  - function
  - pointer

Array

- An array is a series of elements of the same data type placed in contiguous memory location.
- Like regular variable ,an array must be declare before it is used.

Syntax:

```
data_type name[size];
```

Example:

```
char name[10];
int student[10];
```

Function

- A function is a group of statement that together perform a task.
- Every c++ program has at least one function that is main() and programmer can define additional function.
- You must define function prototype before use them.
- A function prototype tells compiler the name of the function ,return type, and parameters.

Syntax

```
return_type function_name(parameter_list);
```

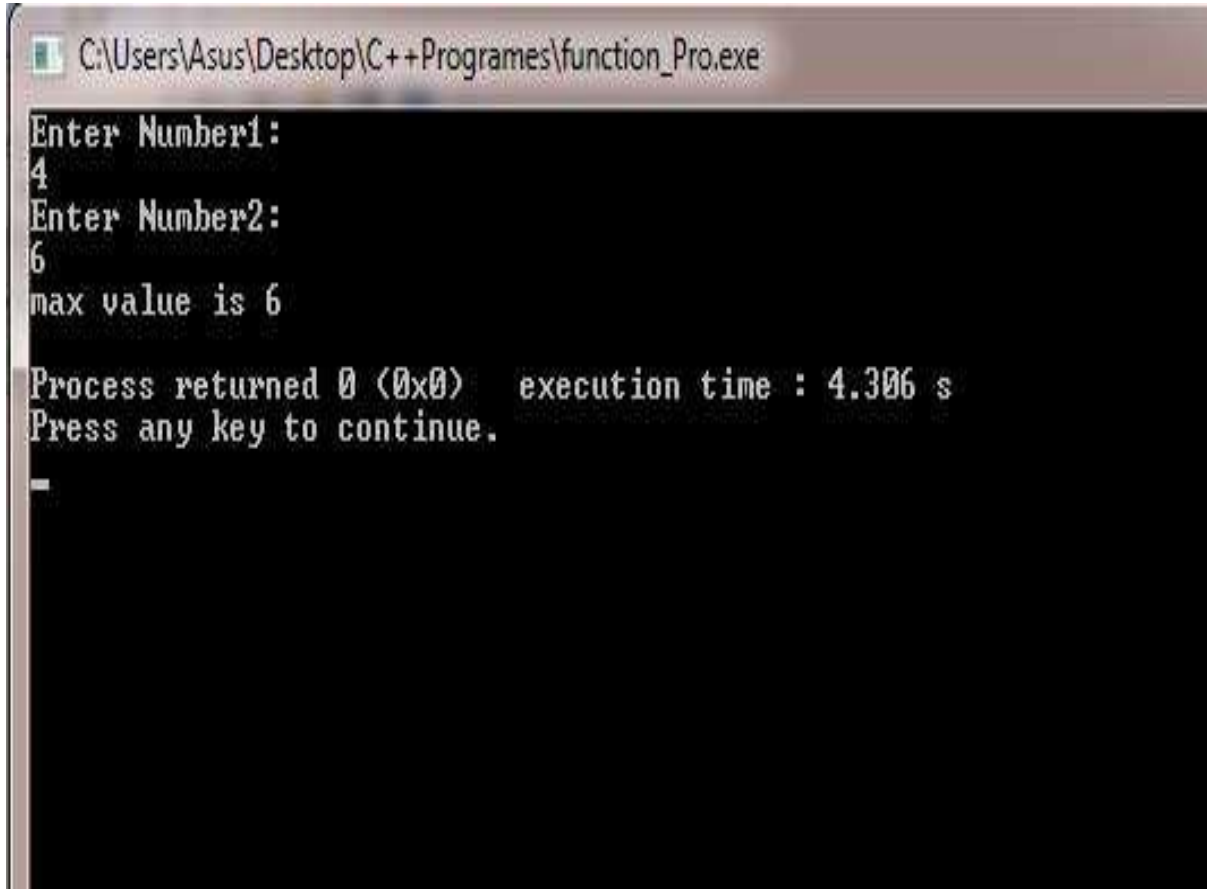
Example

```
int max(int n1,int n2);
```

Function Example

```
#include<iostream>
using namespace  std;

int max(int n1,int n2);                          \\declaration

int main()
{
        int n1;  int n2;  int a;
        cout<<"Enter Number1: "<<"\n";
        cin>>n1;
        cout<<"Enter Number2: "<<"\n";
        cin>>n2;

        a=max(n1,n2);                            \\calling   function

        cout<<"max value is "<<a<<"\n";
        return 0;
}
```

```
int max(int n1,int n2)
{
   int result;

   if(n1>n2)
        result=n1;
   else
        result=n2;

   return result;
}
```


C:\Users\Asus\Desktop\C++Programes\function_Pro.exe
Enter Number1:
4
Enter Number2:
6
max value is 6
Process returned 0 (0x0)   execution time : 4.306 s
Press any key to continue.

Pointer

- Pointer is basically the same as any other variable, difference about them is that instead of containing actual data they contain a pointer to the memory location where information can be found.
- Basically, pointer is variable whose value is address of another variable.

Syntax:

```
type *var_name;
```

Example:

```
int *p;
```

Pointer

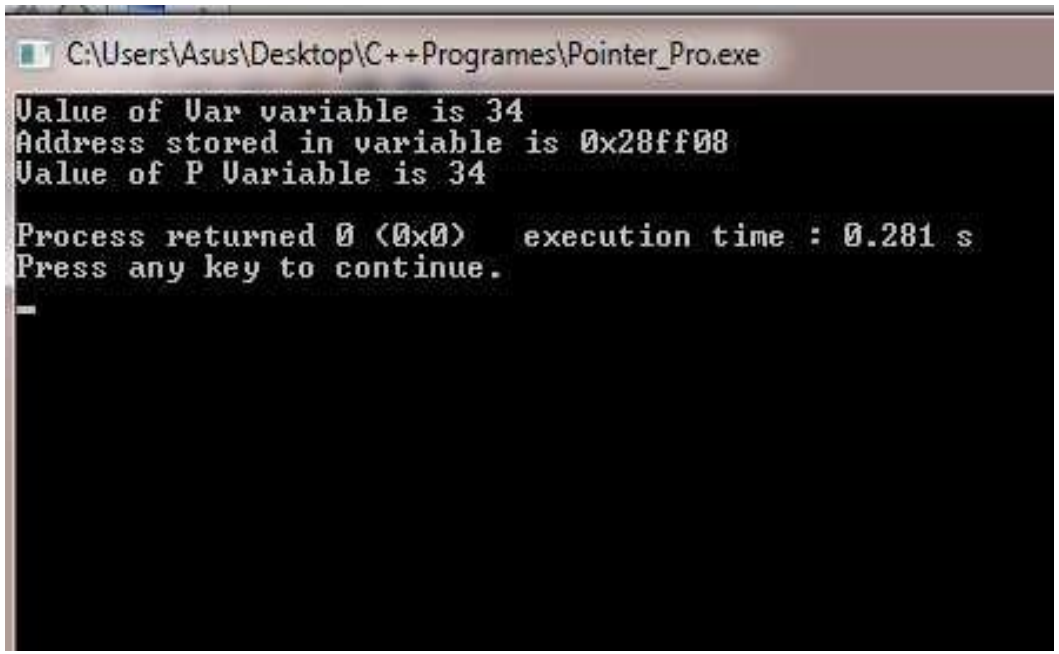
- There are few operation which will do frequently with pointer is:
  - We define a pointer variable.
  - Assign address of variable to a pointer and
  - Finally access the value at the address available in the pointer variable.

Like,

I. int var=20;
II. int *p;
III. p =&var;

Pointer Example

```
#include<iostream>
using namespace std;

int main()
{
	int var=34;
	int *p;
	p=&var;

	cout<<"Value of Var variable is "<<var<<"\n";

	cout<<"Address stored in variable is "<<p<<"\n";

	cout<<"Value of P Variable is "<<*p<<"\n";

	return 0;
}
```



symbolic constant type

- Constant is an identifier with associated value which can not be altered by the program during execution.
- You must initialize a constant when you craete it,you can not assign new value later after constant is initialized.
- Symbolic constant is a constant that is represented by name.

  - **Defining constant with ' #define'.**
  - **Defining constant with 'const'.**
  - **Defining constant with 'enum'.**

Example:

```
#define pi 3.14
const max=10;
enum char{a,b,c};
```

sizeof() operator

- The sizeof is a keyword but it is compile time operator that determine the size, in byte, of a variable or data type.

- It can be used to get the size of classes, structure, union and any other user defined data type.

Syntax:

sizeof(data_type)

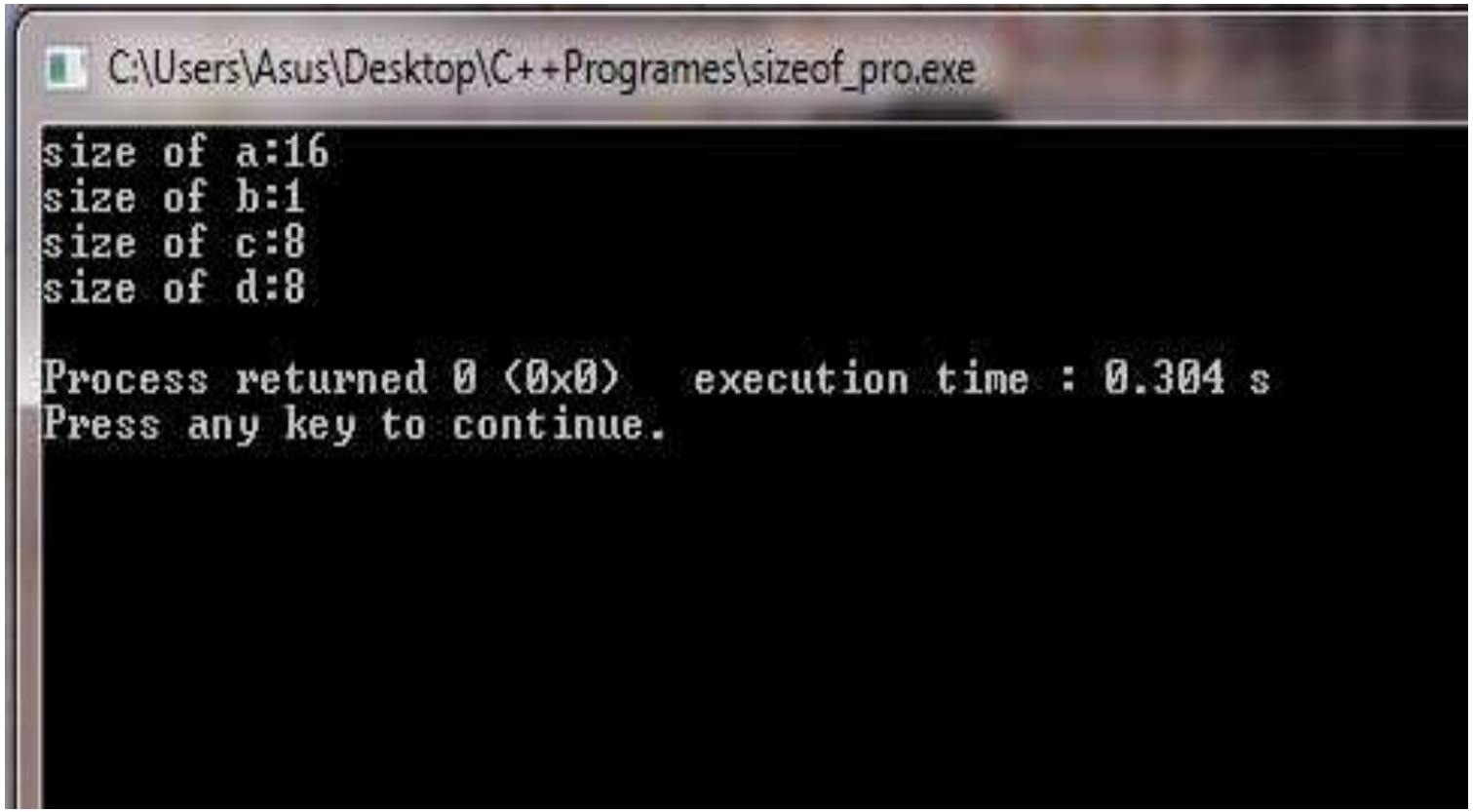
Example:

sizeof(x) or sizeof(int)

- sizeof operater will return integer value.

Example of sizeof operator

```
#include<iostream>
using namespace std;

int main()
{
int a,x;
char b;
float c=10.02;
double d;
cout<<"size of a:"<<sizeof(a)*sizeof(x)<<"\n";
cout<<"size of b:"<<sizeof(b)<<"\n";
cout<<"size of c:"<<sizeof(10.02)<<"\n";
cout<<"size of d:"<<sizeof(d)<<"\n";
return 0;
}
```


C:\Users\Asus\Desktop\C++Programes\sizeof_pro.exe
size of a:16
size of b:1
size of c:8
size of d:8
Process returned 0 (0x0)   execution time : 0.304 s
Press any key to continue.

Challenges related to data type portability

- In C++, data type portability challenges often arise due to the differences in how compilers, platforms, and architectures handle basic data types.

- Since C++ does not explicitly guarantee fixed-size types (other than for those specified by the C++11 <cstdint> standard), the size of basic types like int, long, short, etc., can vary across different systems.

- This can lead to issues when code is intended to run on multiple platforms with varying architectures (e.g., 32-bit vs 64-bit systems, or big-endian vs little-endian machines).

Challenges related to data type portability

Size of Primitive Data Types

- **Problem**: The size of fundamental types like int, long, short, and char is not guaranteed. For example, int could be 32 bits on one platform and 16 bits on another.
- **Solution**: Use fixed-width types from <cstdint> such as int32_t, int16_t, int64_t, uint32_t, etc., to ensure consistent sizes across platforms.

Pointer Size

- **Problem**: On 32-bit systems, pointers are 4 bytes, while on 64-bit systems, they are 8 bytes. This can lead to issues in programs that assume a specific pointer size for calculations or memory management.
- **Solution**: Use uintptr_t or intptr_t from <cstdint> for performing pointer arithmetic or storing pointer values as integers. These types are guaranteed to be able to hold a pointer value across platforms.

Procedural Programming Language

- List of instructions to tell the computer, what to do step by step
- Procedures
- Top-Down Approach
- (Structured- Top to Bottom)

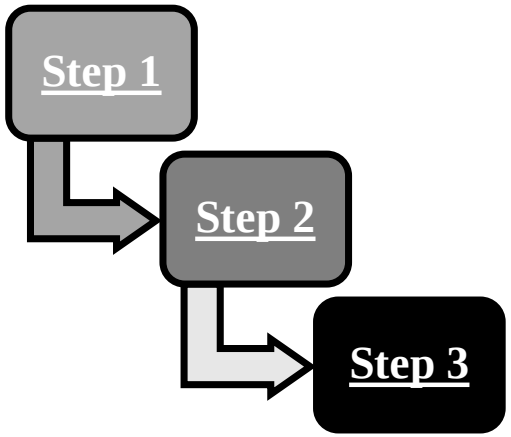


Fig:1 Top-Down Approach

Examples:

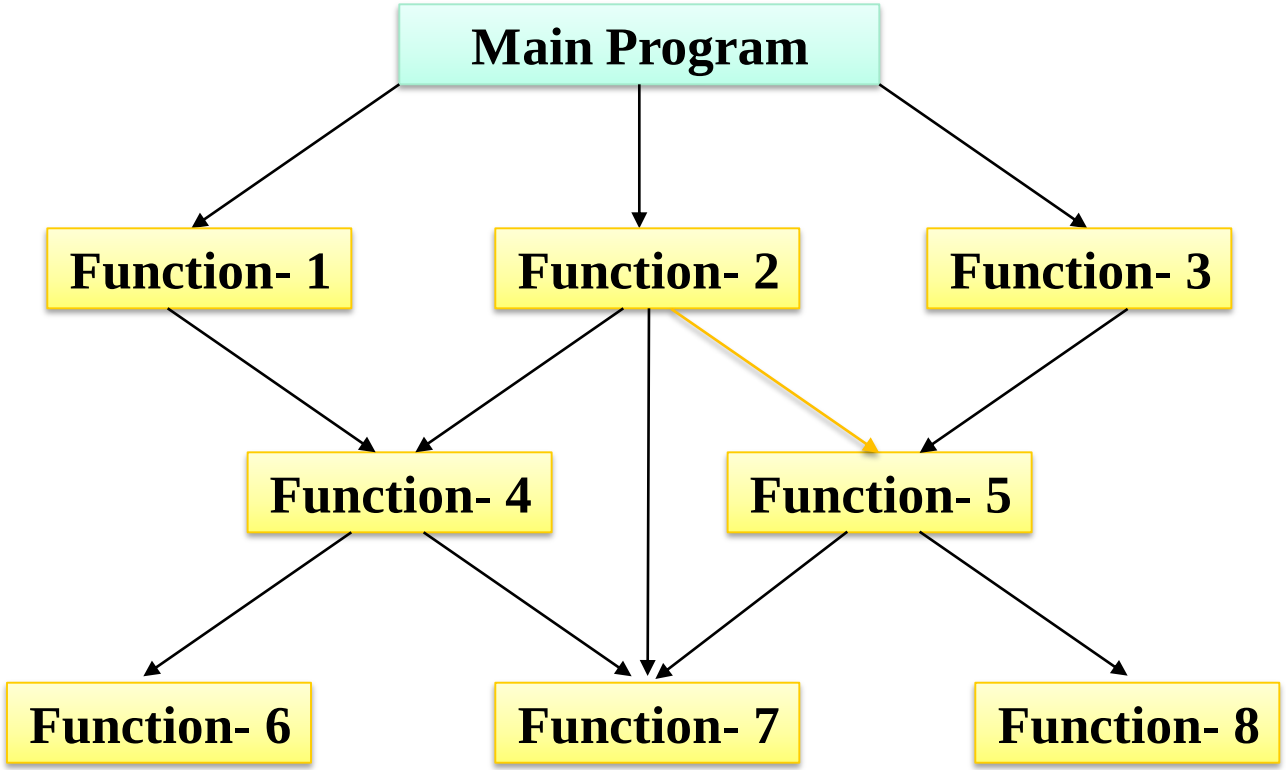
PASCAL

Fortran

THE C PROGRAMMING LANGUAGE

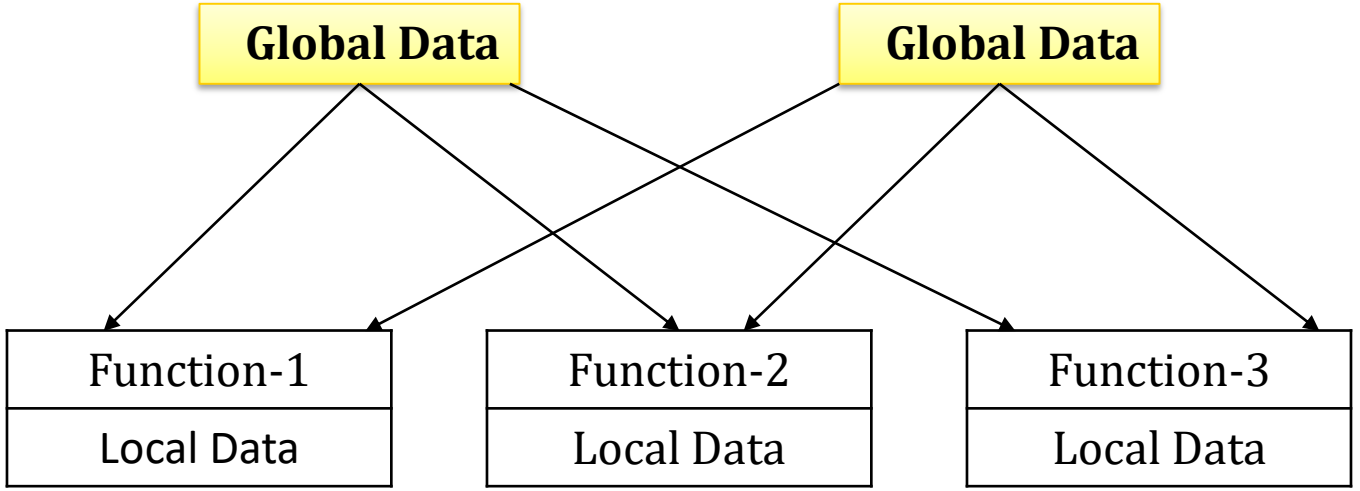
COBOL PROGRAMMING

Structure of Procedure Oriented Programming



Relationship of Data and Function in Procedure Oriented Programming.

- In multi functioned program, many important data items are placed as global so that they may be accessed by all the functions.
- Each function may have its own local data.



Relationship of data and functions in procedural programming

Difference between Procedural Oriented and OOP Programming

Aspect	Procedural Programming	Object-Oriented Programming (OOP)
Focus	Functions and procedures that act on data	Objects that combine data and behavior
Data & Functions	Data and functions are separate	Data and functions are bundled together in objects
Approach	Top-down: Breaking tasks into functions	Bottom-up: Defining objects and their interactions
Code Reusability	Reuse functions, but global data may cause side effects	Reuse objects through inheritance and polymorphism
Flexibility	Harder to extend and modify without affecting the whole code	Easier to extend and modify with minimal changes
Data Security	Data is usually global or passed between functions	Data is encapsulated within objects, protected by access modifiers
Real-World Modeling	Focused on step-by-step instructions	Models real-world objects and their interactions

What is Object Oriented Programming?

IT IS A METHOD OF PROGRAMMING WHERE CODE IS DESIGNED AND BASED ON THE FUNCTIONS AND ATTRIBUTES OF THE OBJECTS.

Contd..

Object-Oriented (OO)

- The term Object-Oriented means that we organize software as a collection of discrete object that incorporate both data structure and behavior are only loosely connected.
- OO approach generally includes four aspects: identity, classification , inheritance and polymorphism.

Objects lie at the heart of object-oriented technology

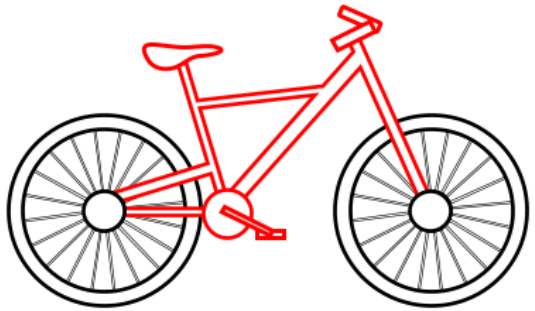
Object:

Examples:

Monitor

Dog

Mango



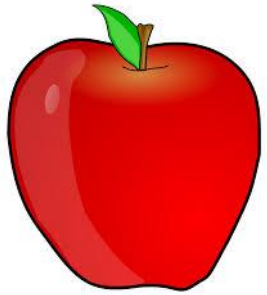
Bicycle

Object Oriented Programming Language:

- **Object:** Entity
- **Examples:**

Table

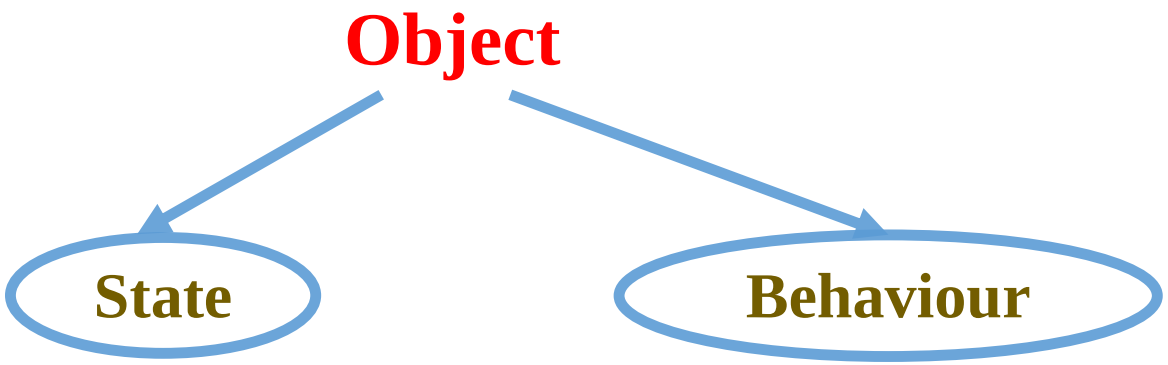
Dog



Apple

Man

Object Oriented Programming Language:



Example:
Dog

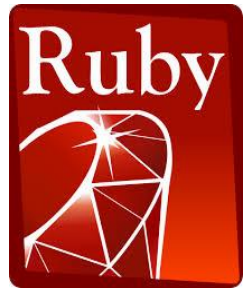
- **Colour**
- **Name**
- **Height**

- **Eating**
- **Barking**
- **Wagging the tail**

Object Oriented Programming Language:

- **Object Oriented: Approach to problem solving where all the problems are carried out using Objects.**
- **Bottom-Up Approach**
- **Starts from basic level of programming feature i.e. class.**

Examples:



Object Oriented Programming Features

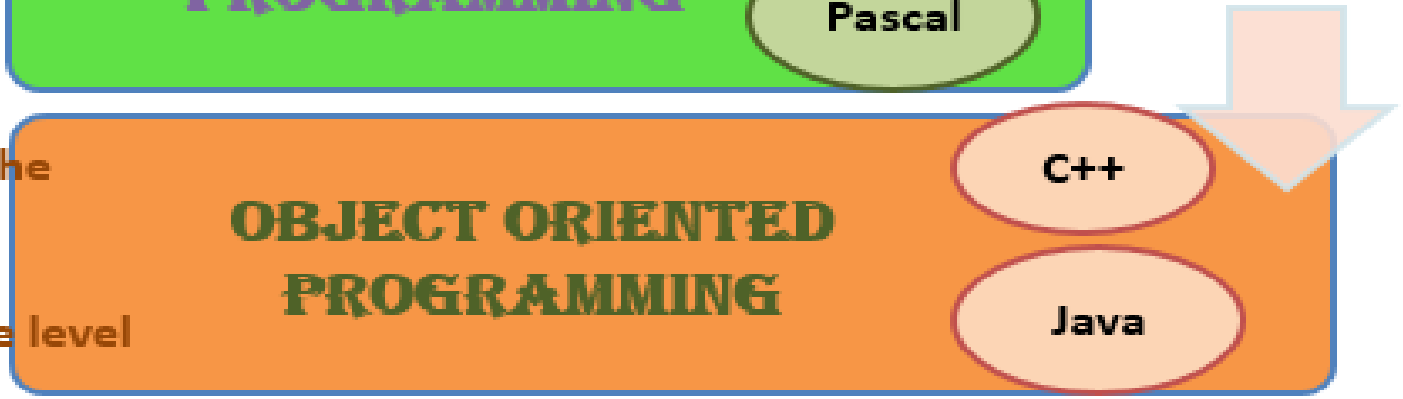
- Emphasis on Data rather than Procedure.
- Programs are divided into Objects.
- Data is Hidden and cannot be accessed by external Function.
- Object may communicate with each other through functions.
- New data and Functions can be added.
- Bottom- Up Approach

Programming Methodologies

- Difficult to interpret
- Difficult to enhance

- Less efficient for enterprise level applications

- Can interact with the other code
- Work on enterprise level applications



Example:

Bank Account Holder withdraws money from ATM.

STEP 1 : INSERT THE ATM.

STEP 2: ENTER THE PIN.

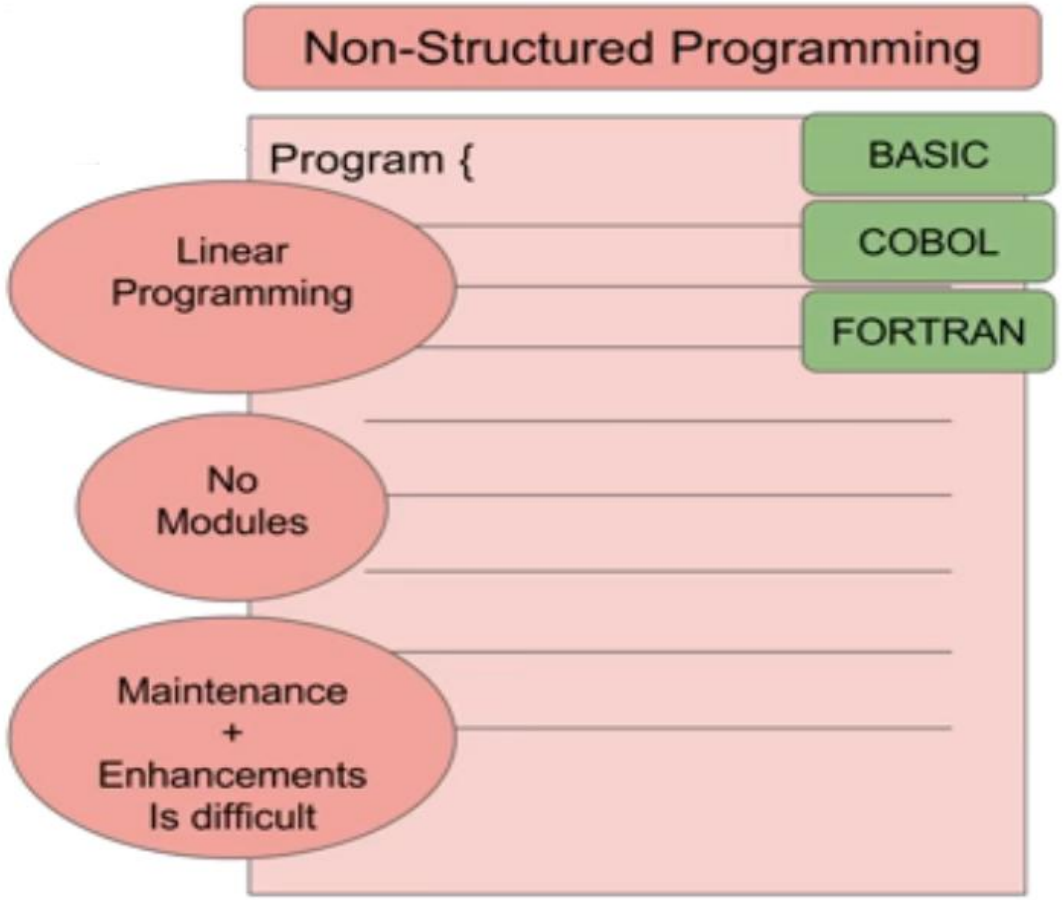
STEP 3: ENTER THE AMOUNT.

STEP 4: WITHDRAW CASH.

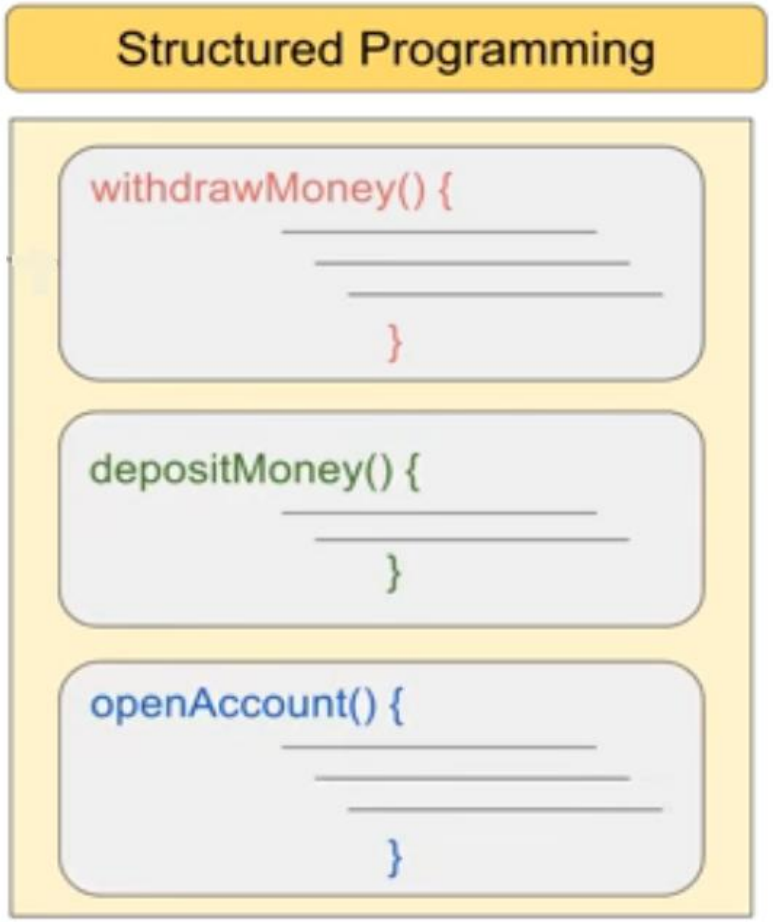
STEP 5: WITHDRAW CARD.

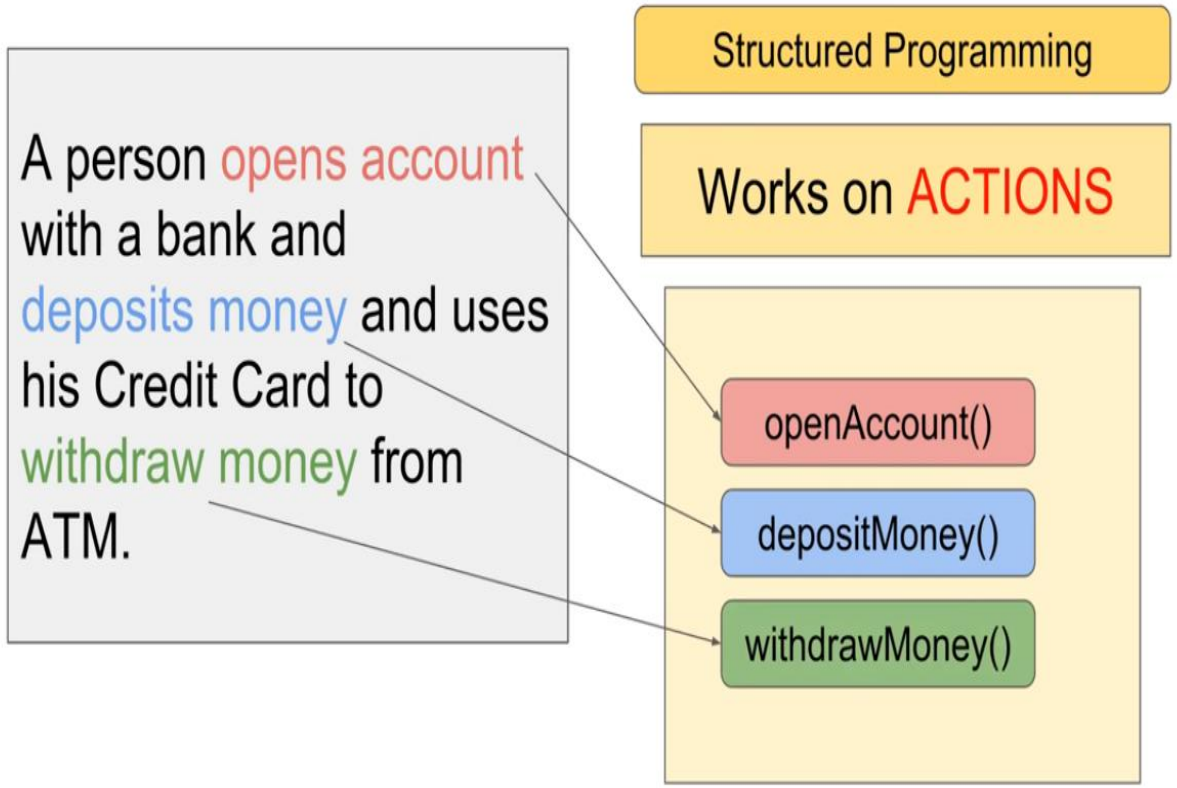
Programming Example using
Non structured methodology

```
10 Input A
15 Input B
20 B = A + 10
30 IF B > 12 GOTO 60
40 C = B / 3
50 IF C < 24 GOTO 10
60 Write C
70 IF Write Failed GOTO 15
80 Input D
```



Programming Example using Structured methodology





Programming Example using
Object Oriented methodology

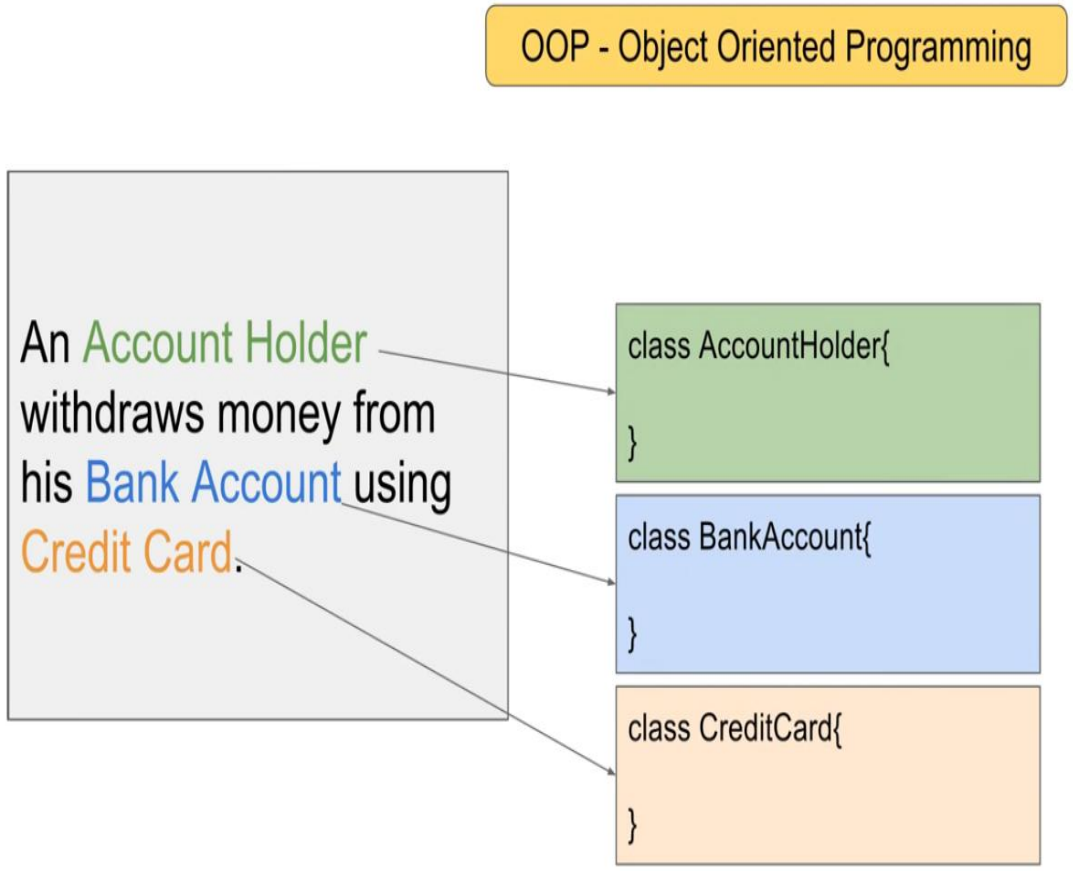
OOP - Object Oriented Programming

Does not work on ACTIONS

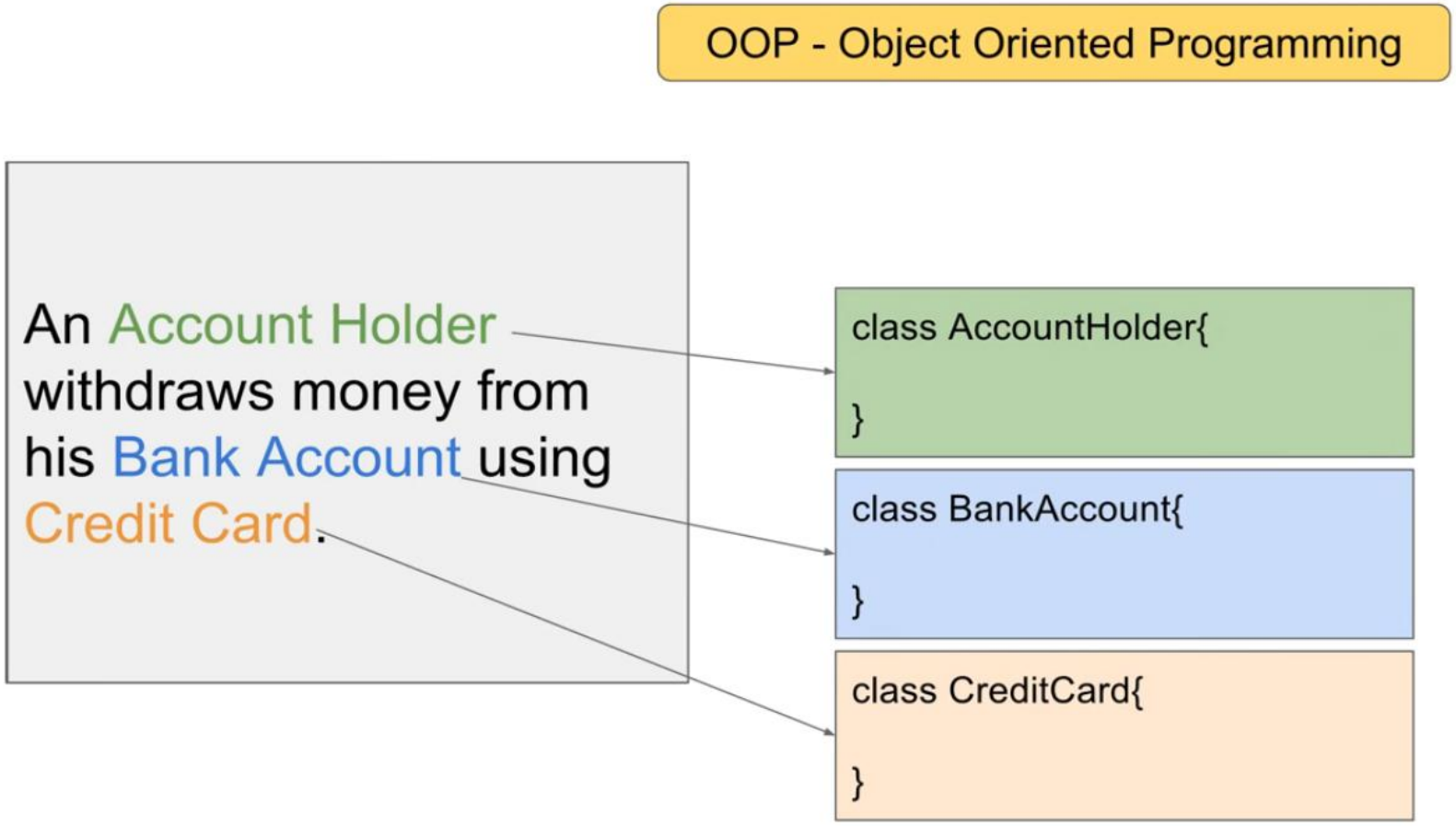
Works With

Classes

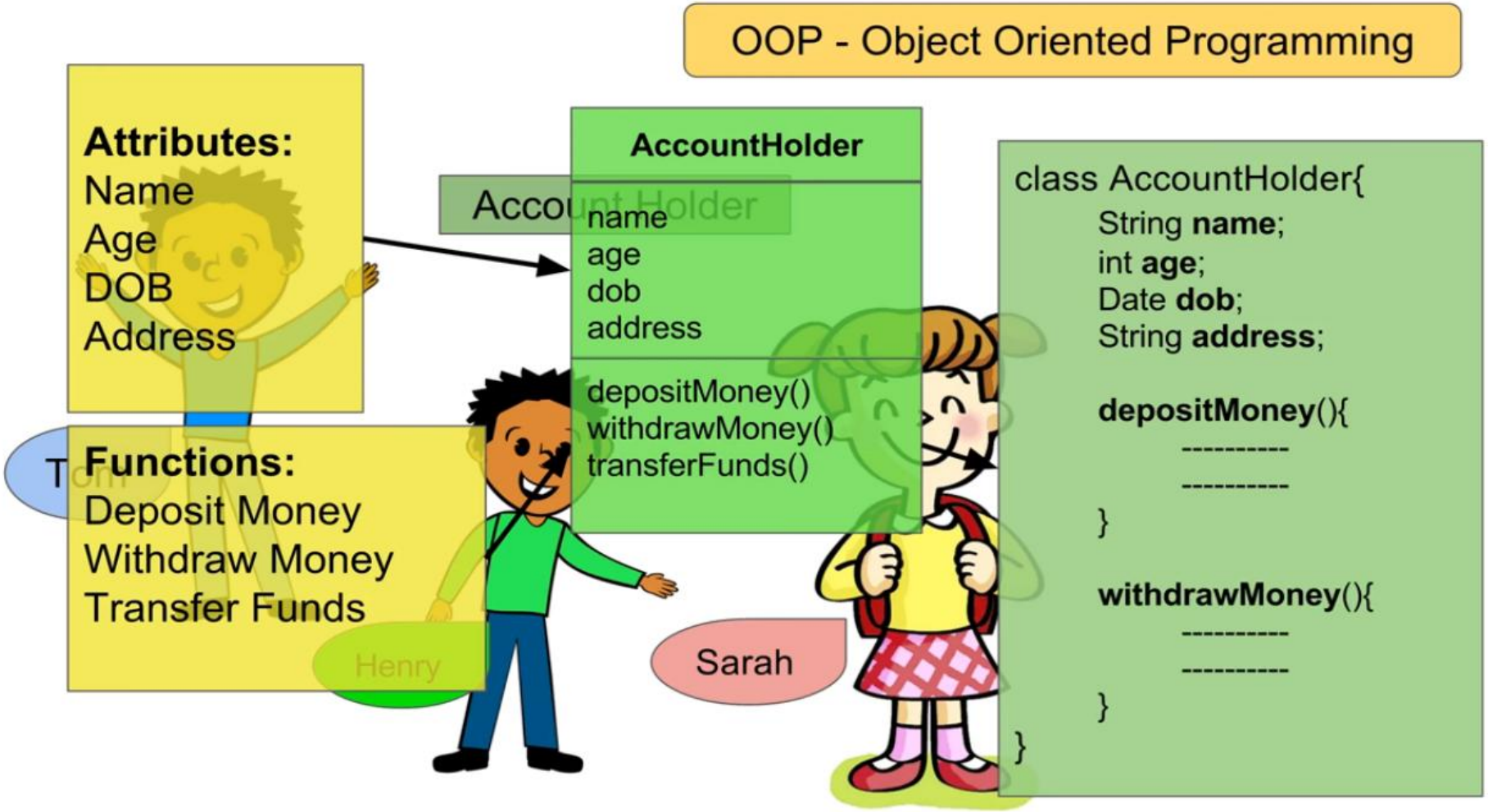
Objects



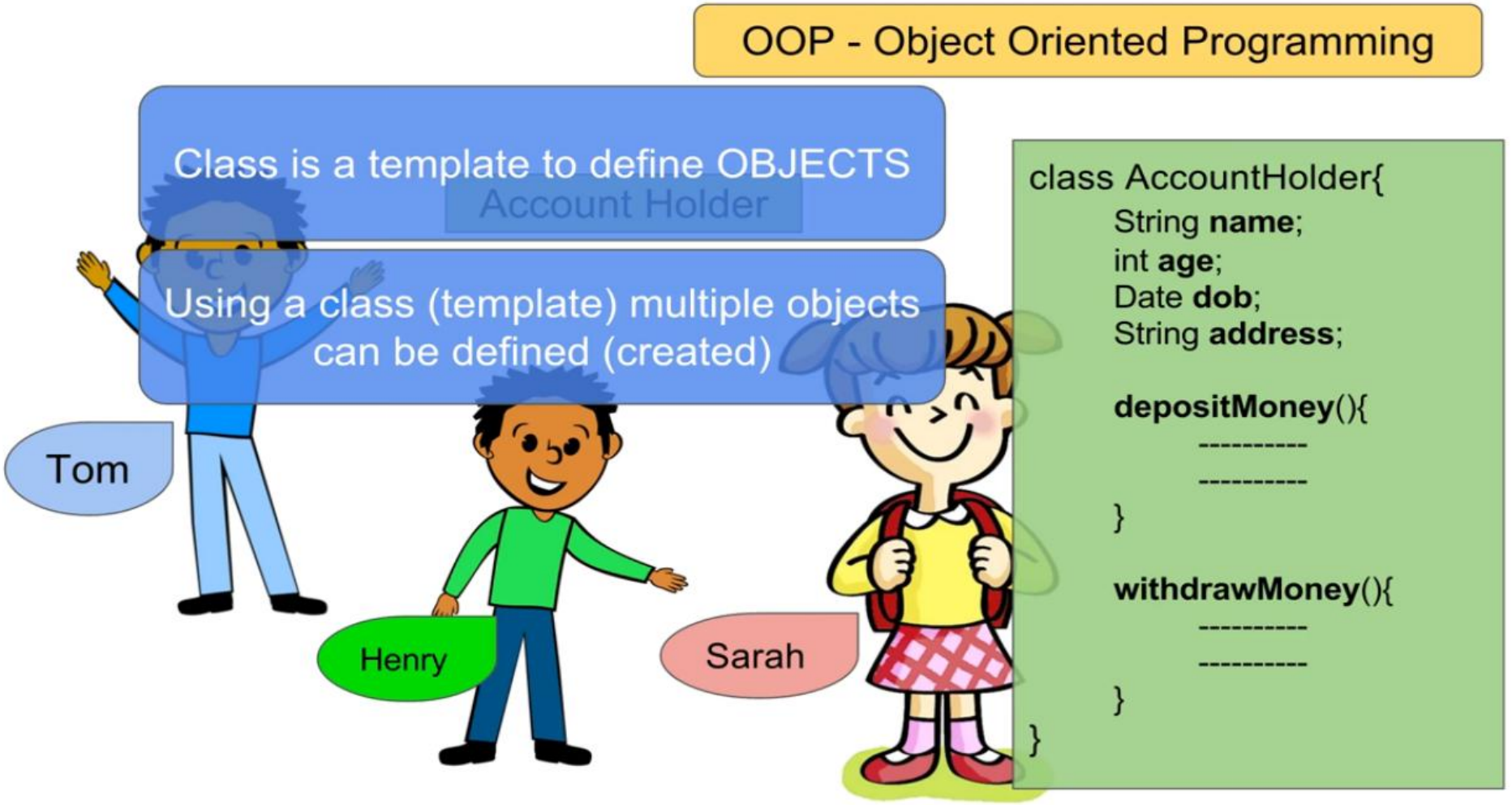
Programming Example using
Object Oriented methodology



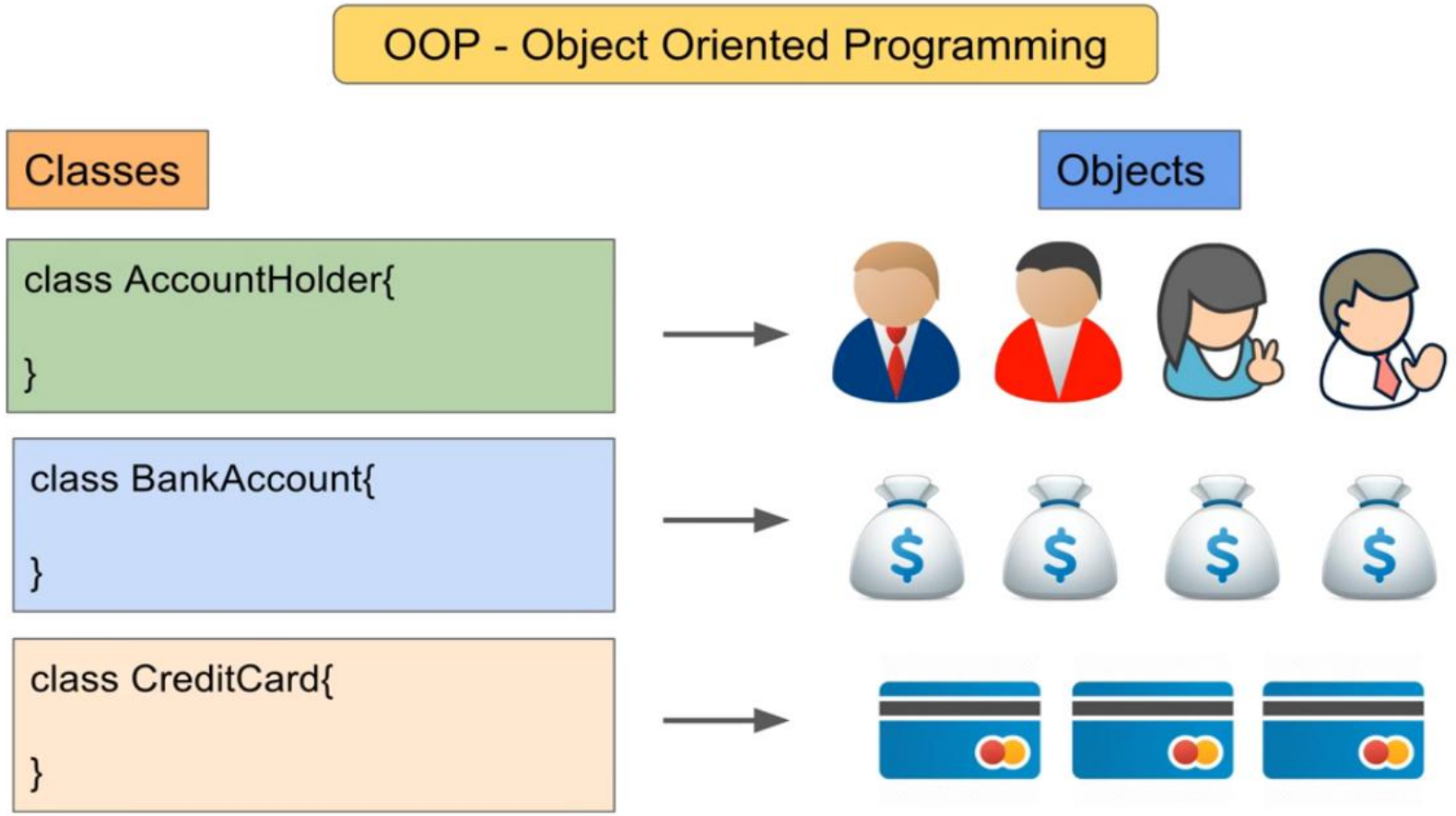
Programming Example using
Object Oriented methodology



Programming Example using
Object Oriented methodology



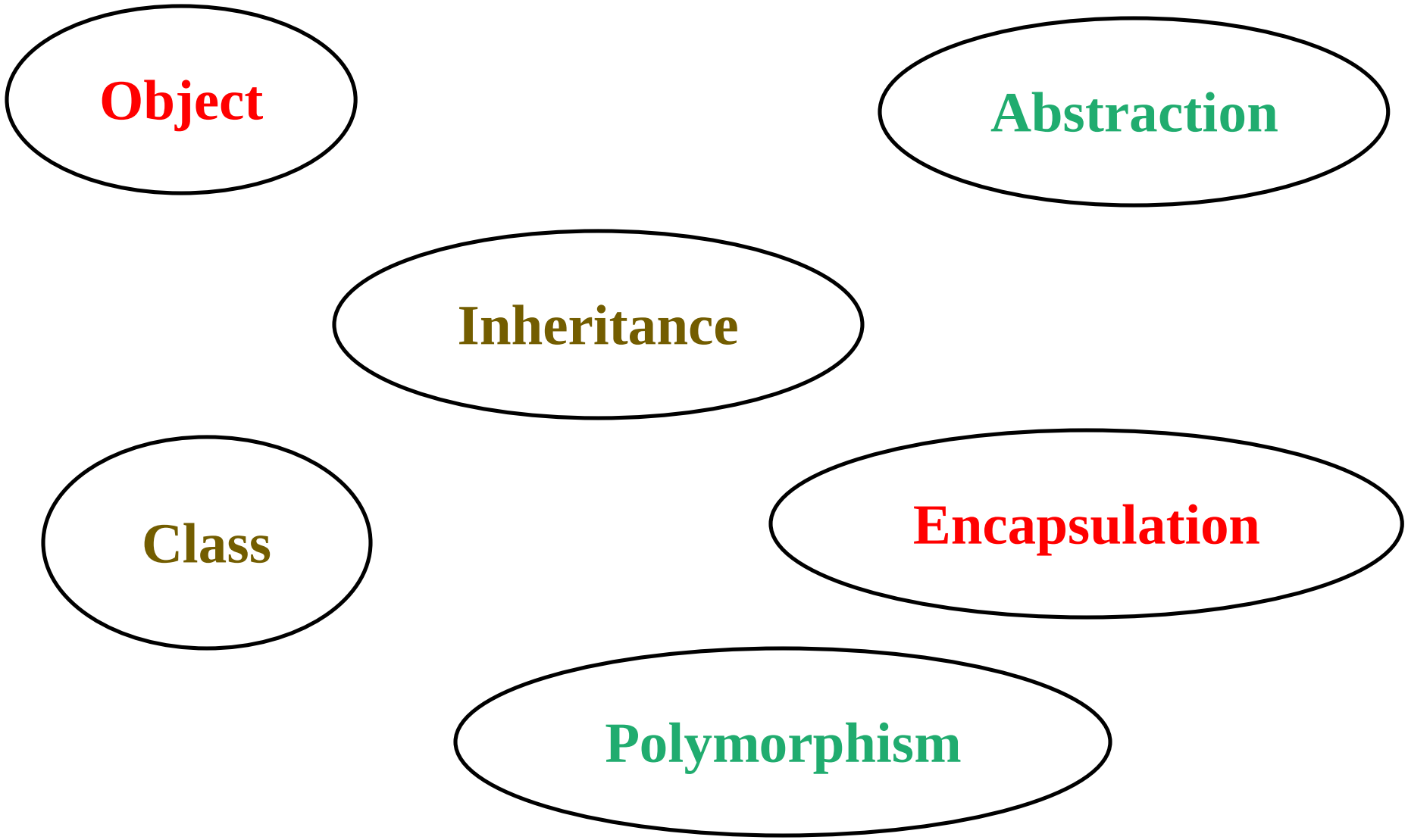
Programming Example using
Object Oriented methodology



Basic Principles of object Orientation

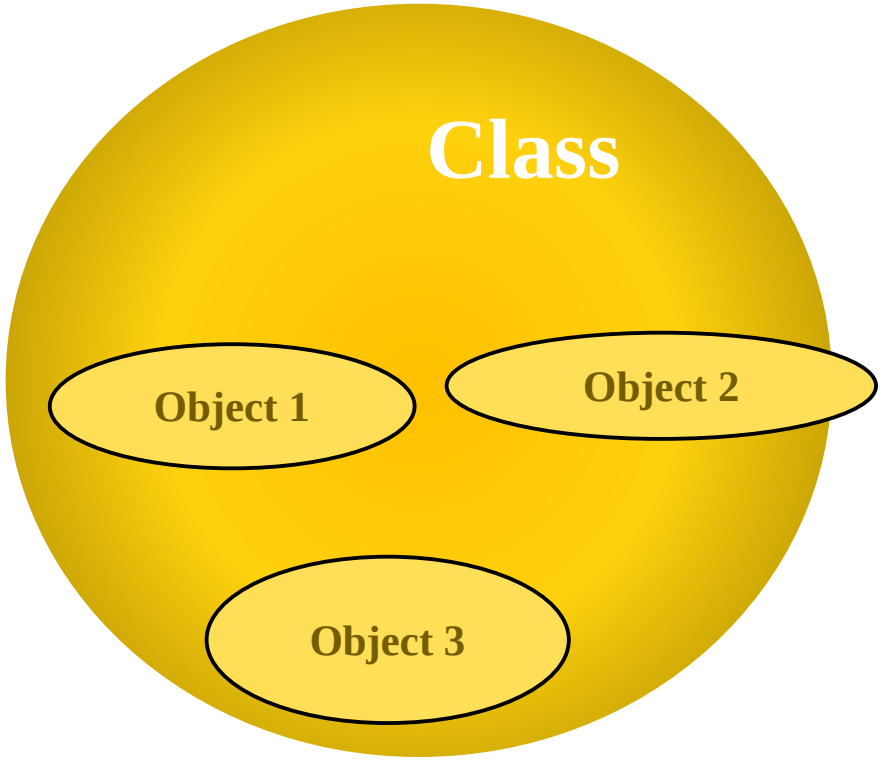
- **Identity:** Qunatized data into discrete, distinguishable entities called objects
- **Classification:** Objects with the same data structure (attributes) and behavior (operations) are grouped into a class.
- **Inheritance:** Sharing or attributes and operations– among classes based on hierarchical relationship.
- **Polymorphism**: Same operation may behave differently for different classes.
- **Encapsulation**: Hiding internal data and providing controlled access via methods.
- **Abstraction**: Simplifying complex systems by providing simple interfaces and hiding detailed implementation.

Concepts of OOP:

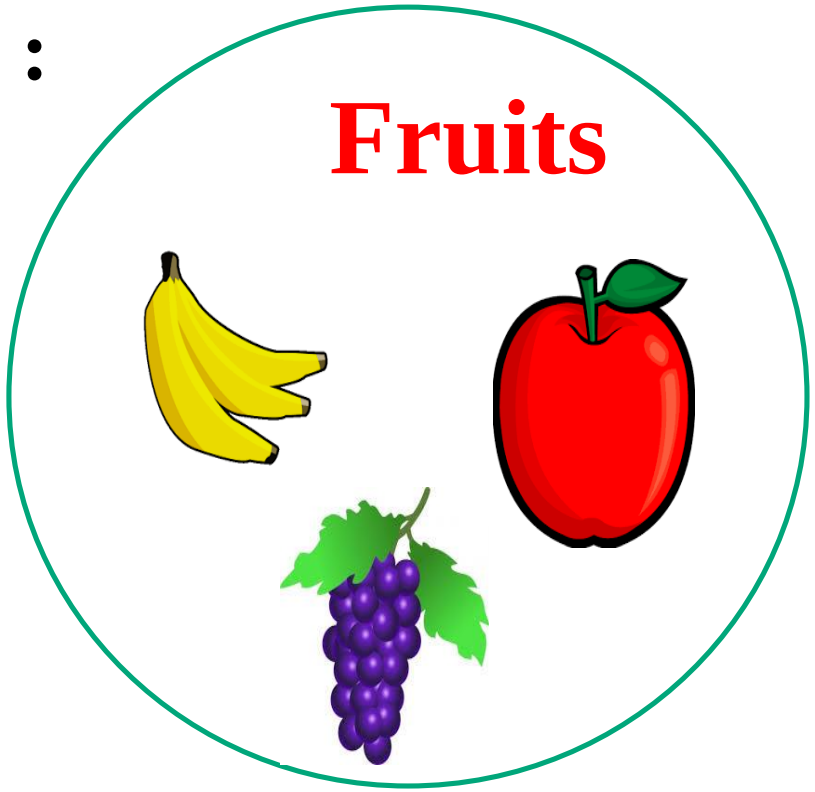


Concepts of OOP:

- **Class: Collection of Objects.**
- **Blueprint of any functional entity which defines its properties and functions**



Example :



What is a Class?

- software “blueprints” for objects are called **classes**

- Definition:
  - A **class** is a blueprint or prototype that defines the variables and methods common to all objects of a certain kind

Class Example

TELEVISION

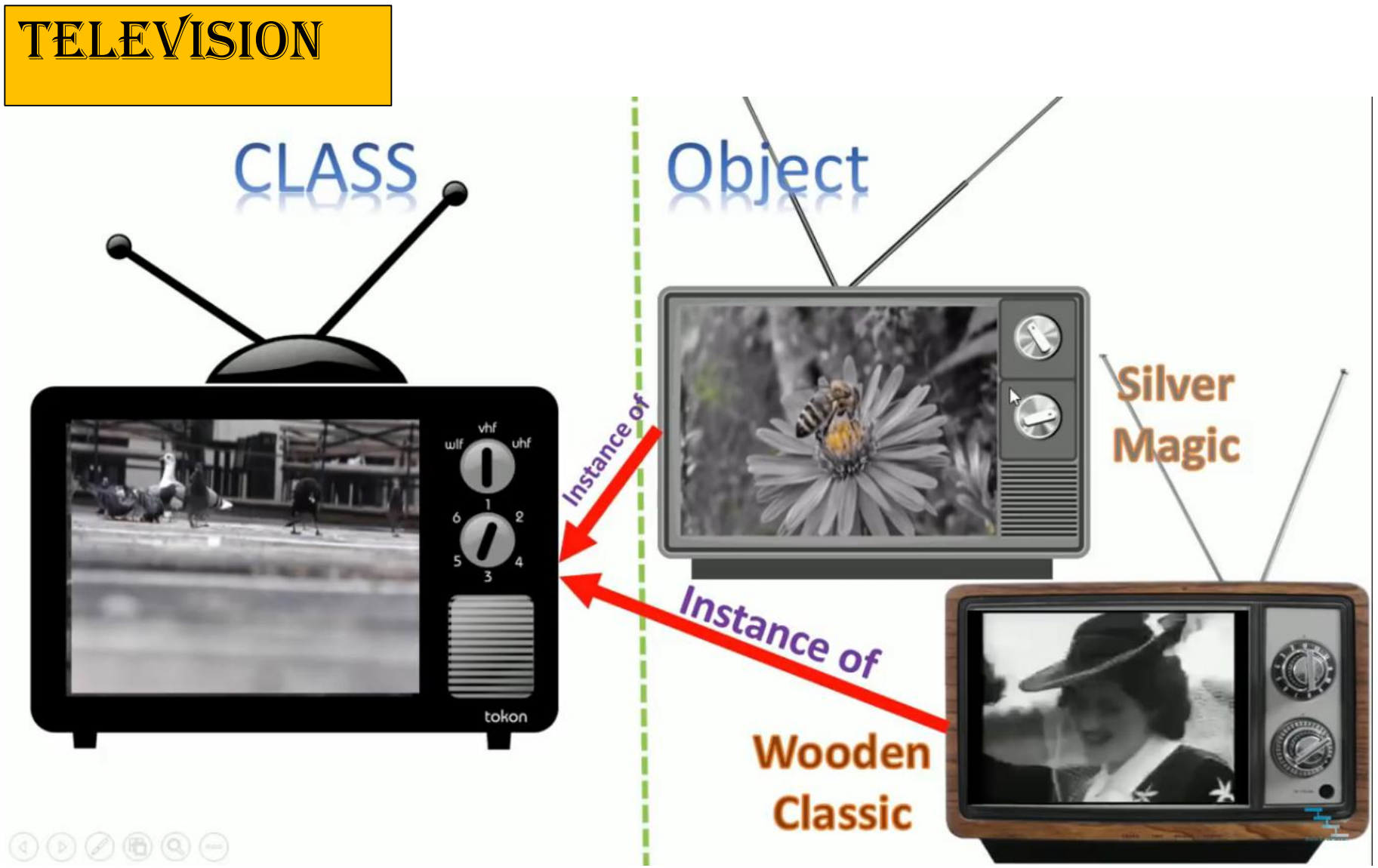
Properties

- Screen Height
- Screen Width
- Screen Shape

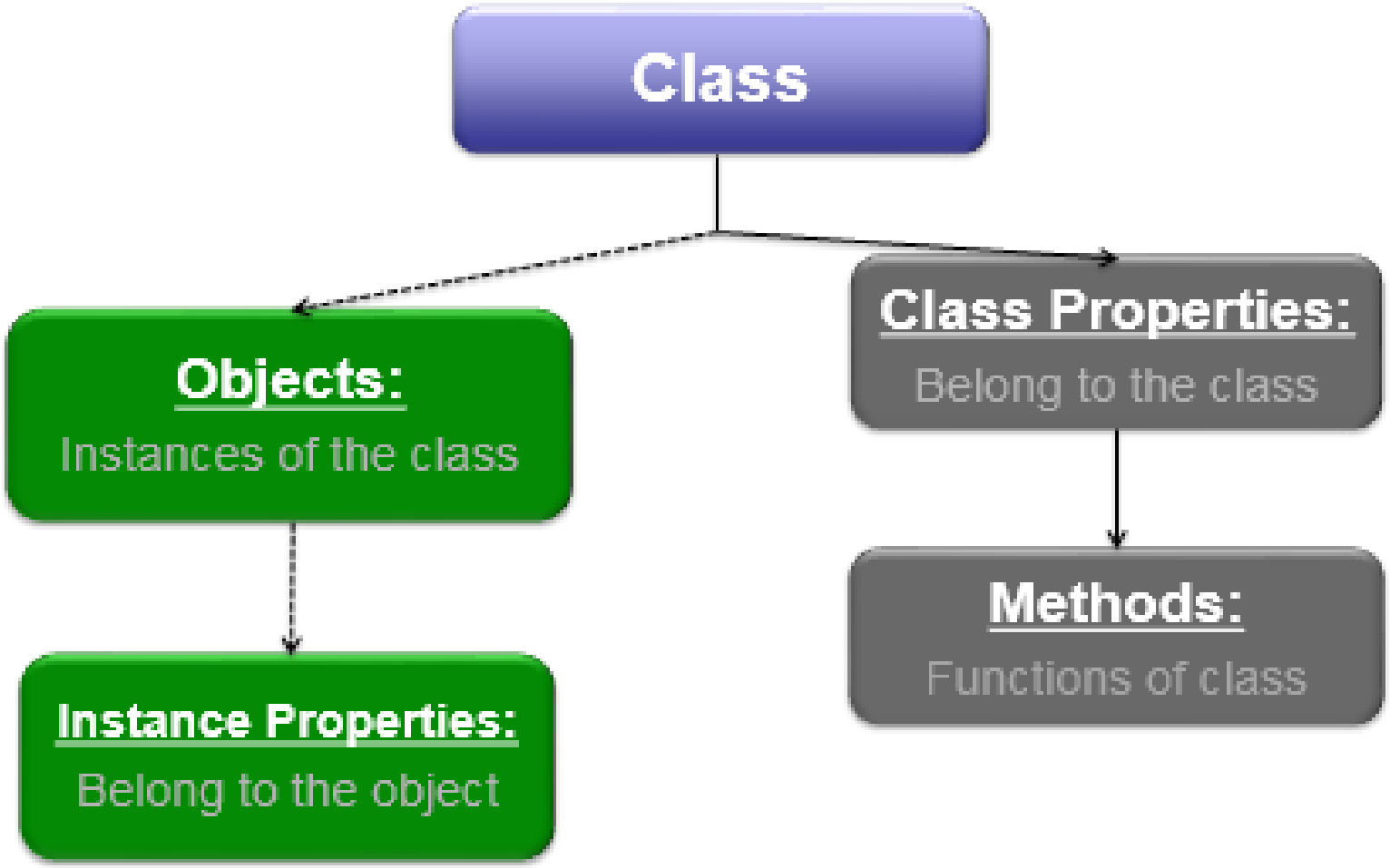
Functionalities

- On Off Switch
- Volume Control
- Tuner

Class and Object with Example



Classes

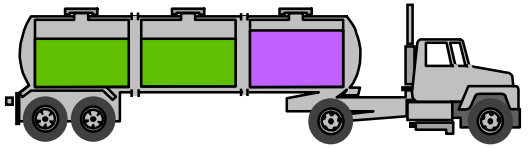

Class
Objects:
Instances of the class
Class Properties:
Belong to the class
Instance Properties:
Belong to the object
Methods:
Functions of class

Almost everything in the world can be represented as an object

- A flower, a tree, an animal
- A student, a professor
- A desk, a chair, a classroom, a building
- A university, a city, a country
- The world, the universe
- A subject such as CS, IS, Math, History, …

More about objects

- Informally, an object represents an entity, either physical, conceptual, or software.
  - Physical entity

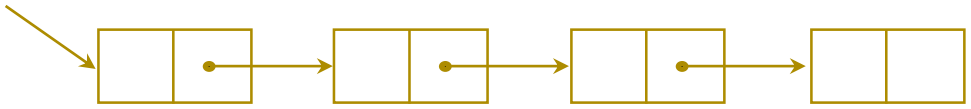


Truck

  - Conceptual entity

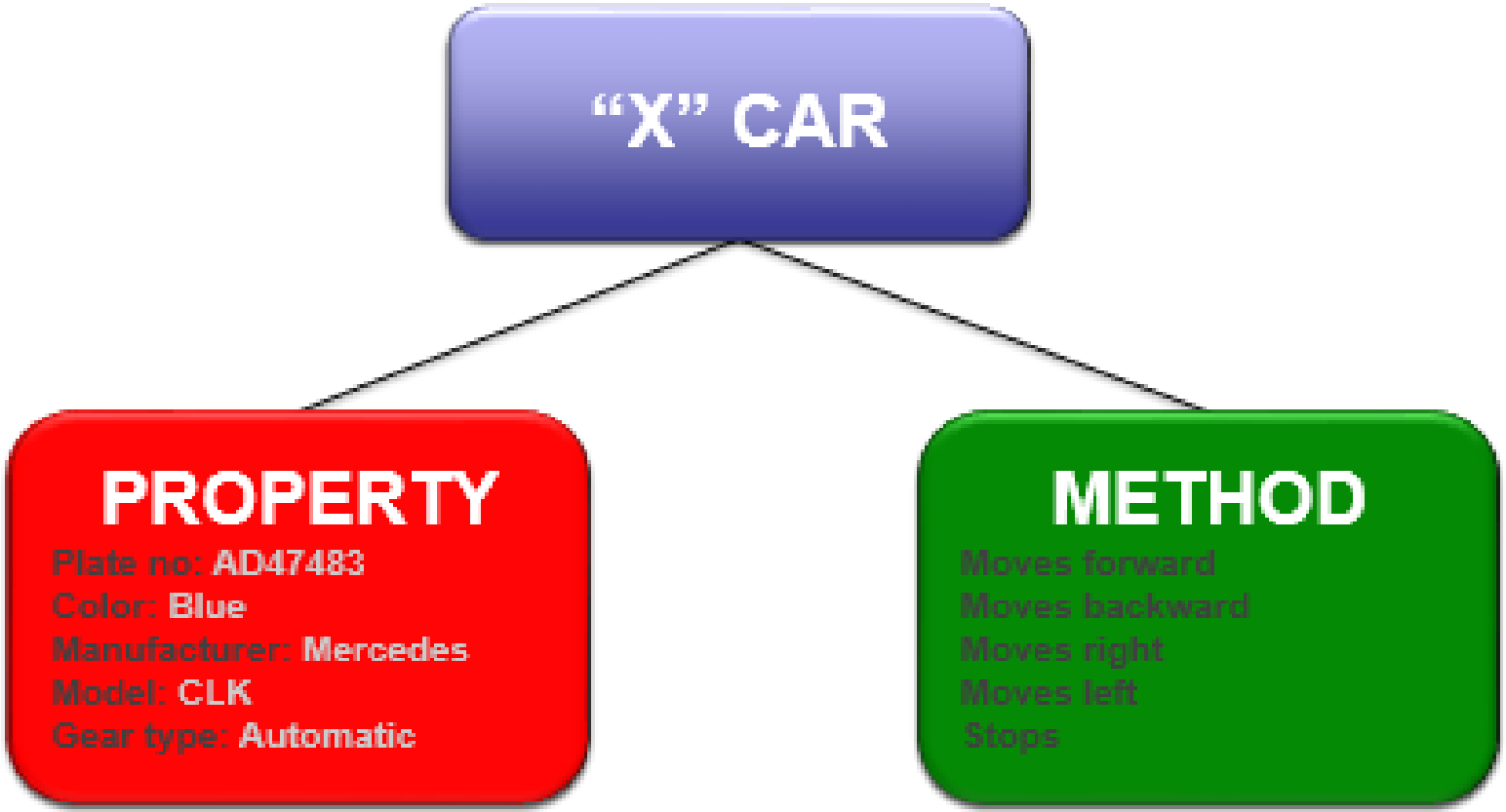
Chemical Process

  - Software entity



Linked List

Classes & Objects



Class/Object

Each copy of an object from a particular class is called an ***instance*** of the class.

Class/Object

The act of creating a new instance of an class is called **instantiation.**

In short…

- An Object is a Class when it comes alive!
- Student is a class, Suresh and Ramesh are objects
- Animal is a class, the cat is an object
- Vehicle is a class, My neighbor's BMW is an object
- Galaxy is a class, the MilkyWay is an object

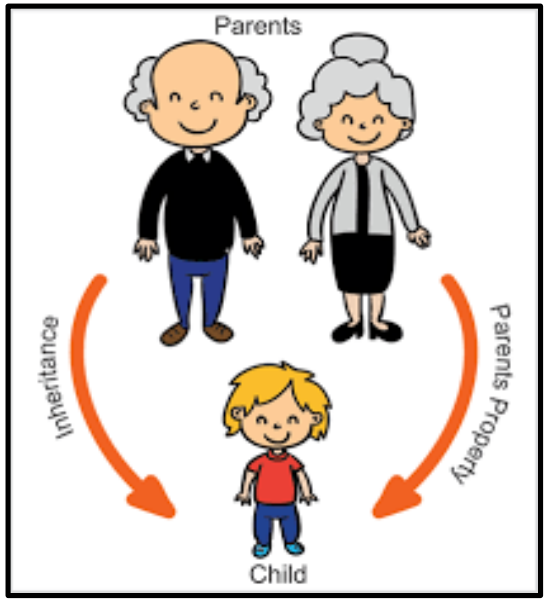
Technical contrast between Objects & Classes

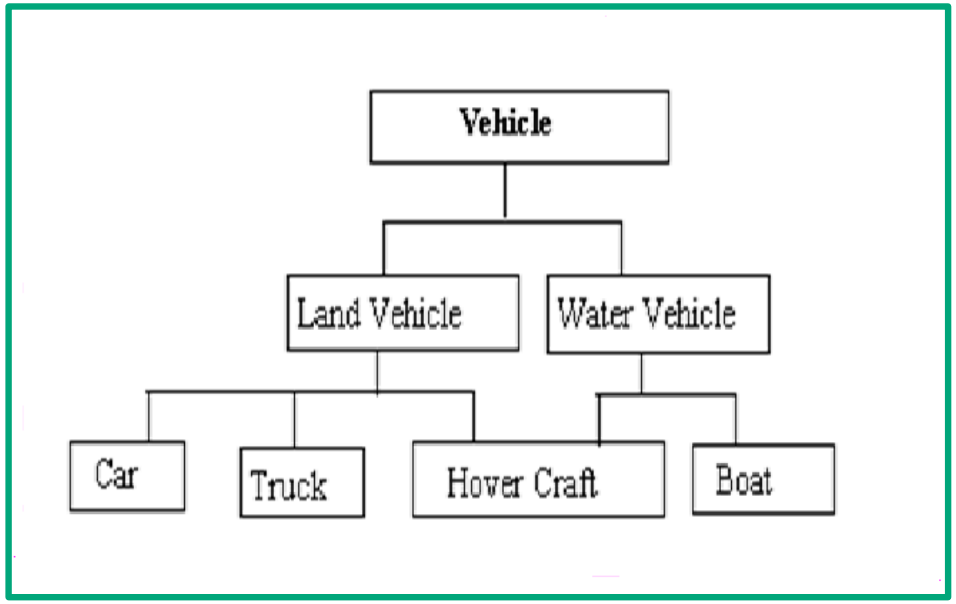
CLASS	OBJECT
Class is a data type	Object is an instance of Class.
It generates OBJECTS	It gives life to CLASS
Does not occupy memory location	It occupies memory location.
It cannot be manipulated because it is not available in memory *(except static class)*	It can be manipulated.

Object is a class in *"runtime"*

Inheritance

- When one object acquires all the properties and behaviours of parent object i.e. known as inheritance.
- It provides code reusability.





Types of Inheritance

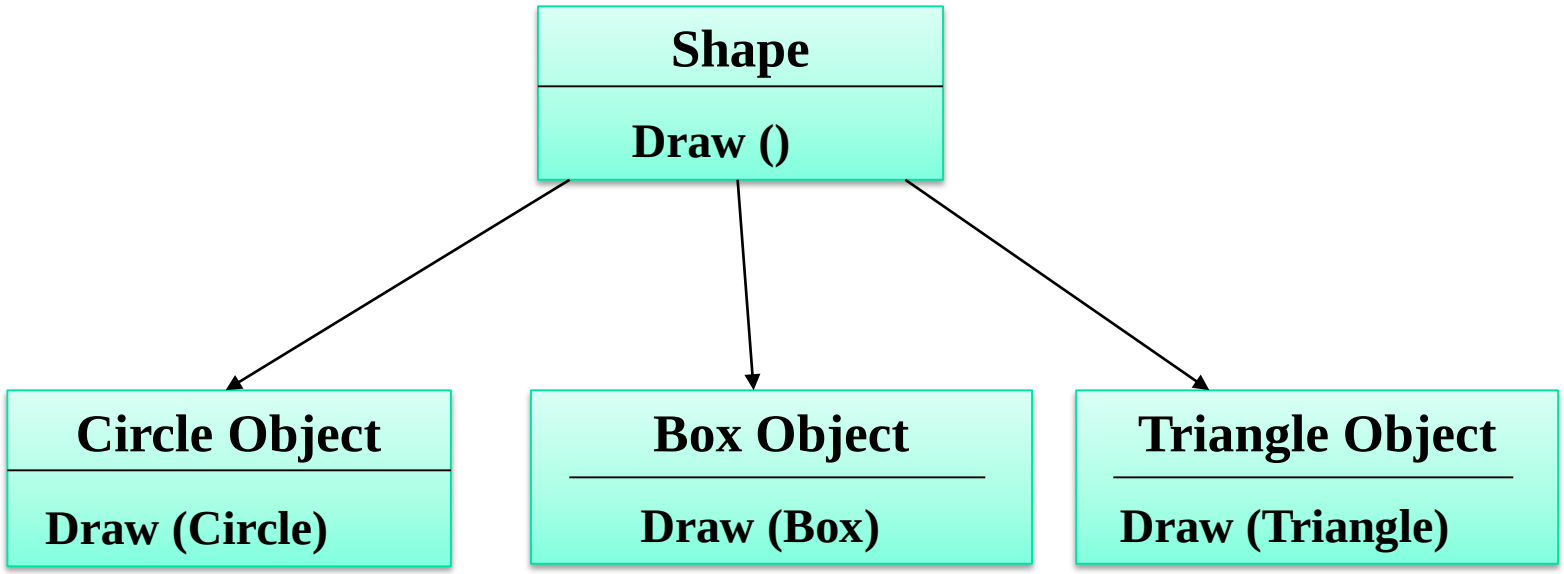
- **Single Inheritance –** A subclass derives from a single super-class.
- **Multiple Inheritance** – A subclass derives from more than one super-classes.
- **Multilevel Inheritance** – A subclass derives from a super-class which in turn is derived from another class and so on.
- **Hierarchical Inheritance –** A class has a number of subclasses each of which may have subsequent subclasses, continuing for a number of levels, so as to form a tree structure.
- **Hybrid Inheritance** – A combination of multiple and multilevel inheritance so as to form a lattice structure.

Polymorphism

- When one task is performed by different ways i.e. known as polymorphism.
- For example:
  - To draw something e.g. shape or rectangle etc.
  - To speak something e.g. cat speaks meaw, dog barks woof etc.

Polymorphism

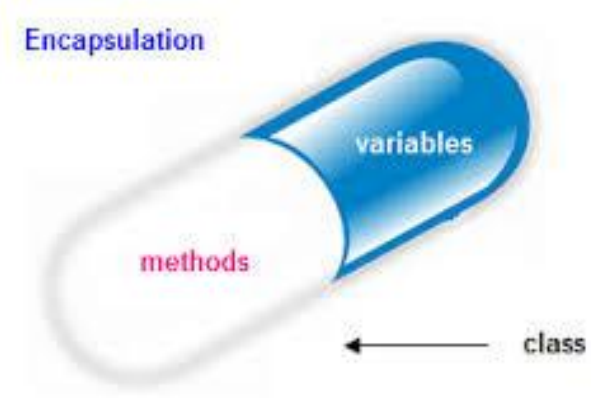
- Single function name can be used to handle different number and different kind of arguments. This is something similar to a particular word having different meaning.
- Using a single function name to perform different types of tasks is known as function overloading.



Polymorphism

Encapsulation

- Mechanism of wrapping the data (variables) and code acting on the data (methods) together as a single unit.



- The variables of a class will be hidden from other classes, and can be accessed only through the methods of their current class.
- Also known as data hiding.
- For example: capsule, it is wrapped with different medicines.

Abstraction

- Hiding internal details and showing functionality is known as abstraction.
- For example: phone call, we don't know the internal processing.

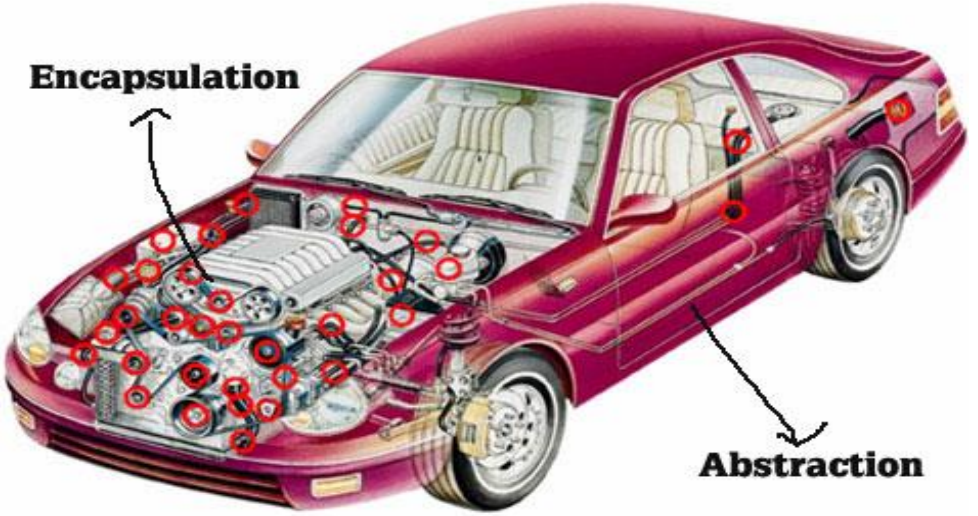
Real Life Example of Abstraction

Data Abstraction

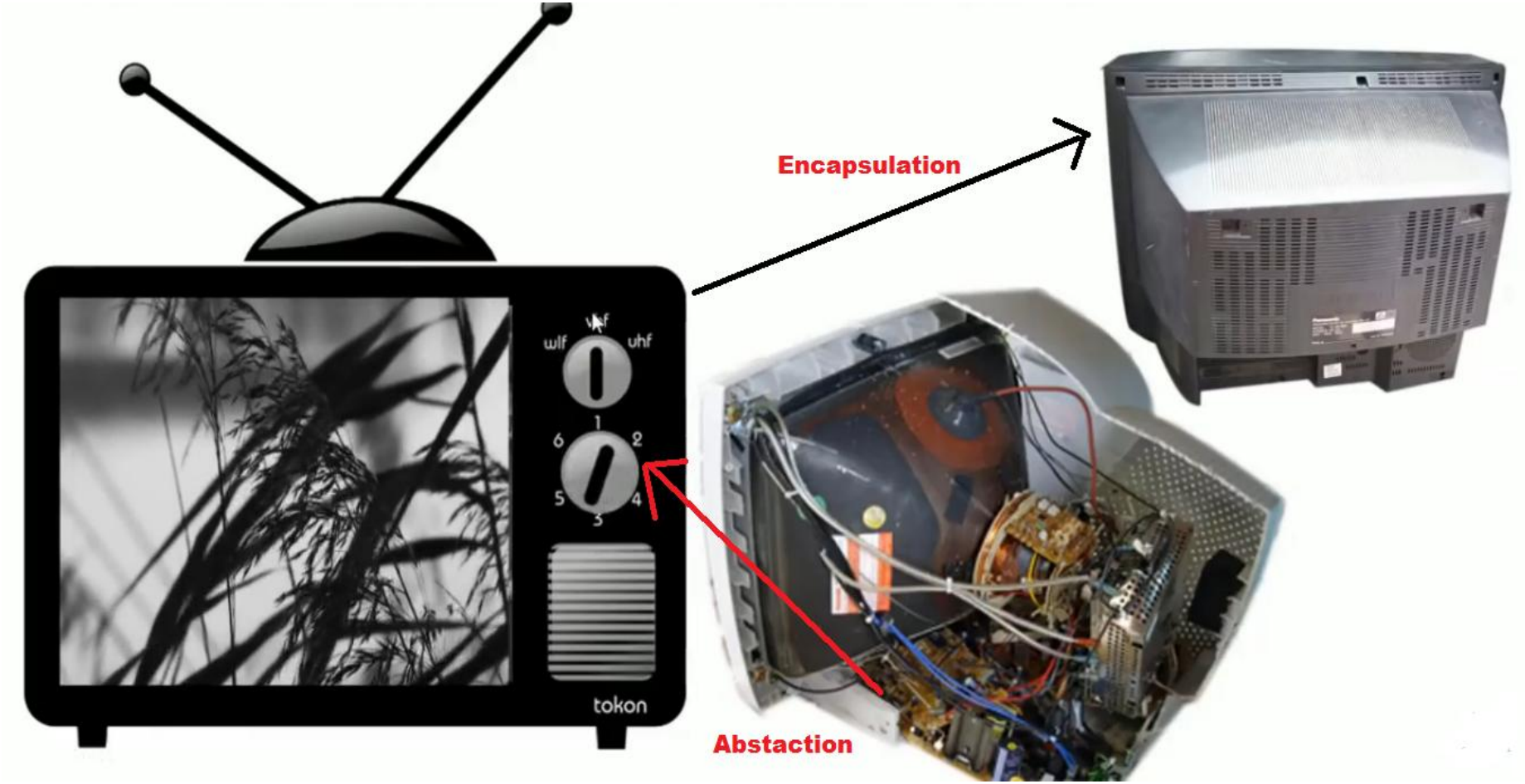
- **Data Abstraction:** Abstraction refers to the act of representing essential features without including the background details or explanation.
- Classes encapsulate all the essential properties of the objects that are to be created.
- The attributes are sometimes called data members, because they hold information.
- The functions that operate on these data are sometimes called methods.

Whether the Encapsulation and Abstraction are same or different?

- **Encapsulation**: Hiding unnecessary things from outside world is known as encapsulation
- **Abstraction**: Displaying only necessary things to the outside world is known as abstraction



Example of Encapsulation and Abstraction



Overloading

- It allows an object to have different meanings depending on the context.
- 2 types:
  1. Operator Overloading
  2. Function Overloading

Overloading is one type of polymorphism

Applications of OOP

- Real-Time System
- Simulation & Modeling
- Object Oriented Database
- Hypertext and hypermedia
- AI and expert system
- Neural Network & Parallel Programming
- Decision Support and Office Automation System
- CIM/CAM/CAD System

Benefits of Object Model

The benefits of using the object model are –

- It helps in faster development of software.
- It is easy to maintain. Suppose a module develops an error, then a programmer can fix that particular module, while the other parts of the software are still up and running.
- It supports relatively hassle-free upgrades.
- It enables reuse of objects, designs, and functions.
- It reduces development risks, particularly in integration of complex systems.

The significance of OOP in software development

- **Code organization**: By organizing code into objects, OOP improves structure and clarity.
- **Code reuse and maintenance**: Inheritance, polymorphism, and encapsulation allow developers to build flexible and reusable code that is easy to extend and modify.
- **Modularity**: It helps break down complex problems into smaller, manageable parts, improving collaboration and team development.
- **Security and Data Integrity**: OOP encapsulates data and exposes only necessary methods, ensuring that the system's state is protected and easily controlled.
- **Real-world modeling**: OOP makes it easier to model real-world systems, enhancing both development and understanding.

Introduction to the C++ Standard Library

The C++ Standard Library is a collection of pre-defined classes, functions, and objects that facilitate common programming tasks like:

- **String manipulation** (std::string)
- **Container management** (std::vector, std::map, etc.)
- **Input/Output** (std::cin, std::cout)
- **Algorithms** (std::sort, std::find)
- **Memory management** (std::unique_ptr, std::shared_ptr)
- **Multithreading** (std::thread, std::mutex)

Namespaces in C++

- A **namespace** in C++ is a feature that organizes code into logical groups and helps prevent naming conflicts.
- As software systems grow larger, it's common for different parts of the program or external libraries to use the same identifiers (e.g., variable names, function names, or class names).
- Namespaces ensure these identifiers can coexist without conflict by placing them in separate "scopes."

```
#include <iostream>

// Custom namespace
namespace MyNamespace {
    int value = 42;
    void display() {
        std::cout << "Value: " << value << std::endl;
    }
}
```

```
int main() {
    // Accessing elements in the namespace
    MyNamespace::display();
    return 0;
}
```

Importance of Using Namespaces

- **Avoiding Naming Conflicts:** Prevents clashes between identifiers (functions, variables, etc.) in large projects or when integrating libraries.
- **Logical Code Organization:** Groups related functionalities together, making the code more structured and manageable.
- **Improved Readability:** Makes it clear which scope an identifier belongs to, aiding understanding.
- **Promoting Modularity:** Enables dividing code into independent, reusable modules.
- **Extensibility:** Allows adding new features to existing namespaces without interfering with the current code.
- **Resolving Ambiguity:** Clarifies which version of an identifier to use when multiple libraries define the same name.
- **Reusability:** Facilitates using code across different projects without naming conflicts.
- **Compatibility with the Standard Library:** Standard Library features (like std::cout) rely on namespaces.
- **Anonymous Namespaces:** Restricts visibility of identifiers to a single file, improving encapsulation.
- **Scalability in Large Projects:** Makes large systems manageable by dividing them into logical scopes.
- **Best Practices Enforcement:** Helps avoid global namespace pollution, ensuring cleaner and conflict-free code.

Structure of a C++ Program

A C++ program typically consists of:

- Preprocessor Directives (e.g., #include, #define).
- Namespace Declaration (optional, e.g., using namespace std;).
- Main Function: The entry point (int main()).
- Class Definitions and Functions: Contain the program's logic.Return Statement:
- The main() function returns an integer to the operating system.

Example

```
#include <iostream>  // Preprocessor directive
using namespace std; // Namespace declaration

class Example {
public:
   void greet() {
      cout << "Welcome to C++!" << endl;
   }
};

int main() {
   Example obj; // Create an object
   obj.greet(); // Call a member function
   return 0;    // Return success status
}
```

Data Members and Member Functions

Data Members

- Variables defined within a class.
- Store data for objects of the class.

Member Functions

- Functions defined within a class to operate on data members.
- Can be defined inside or outside the class.

Example

```
#include <iostream>
using namespace std;

class Person {
private:
    string name;  // Data member
    int age;      // Data member

public:
    void setDetails(string n, int a) { // Member function
        name = n;
        age = a;
    }

    void displayDetails() { // Member function
        cout << "Name: " << name << ", Age: " << age << endl;
    }
};
```

```
int main() {
    Person p;
    p.setDetails("Alice", 25);
    p.displayDetails();
    return 0;
}
```

Initializing Objects with Constructors

A **constructor** is a special member function used to initialize objects. It has the same name as the class and no return type.

Types of Constructors:

- **Default Constructor:** No parameters.
- **Parameterized Constructor:** Accepts parameters.
- **Copy Constructor:** Creates a copy of an object.

Example

```
#include <iostream>
using namespace std;

class Rectangle {
private:
    int width, height;

public:
    // Parameterized constructor
    Rectangle(int w, int h) {
        width = w;
        height = h;
    }

    int area() {
        return width * height;
    }
};
```

```
int main() {
    // Object initialized using the constructor
    Rectangle rect(10, 20);
    cout << "Area: " << rect.area() << endl;
    return 0;
}
```

#include <iostream>

- Lines beginning with a hash sign (#) are directives read and interpreted by what is known as the *preprocessor*. They are special lines interpreted before the compilation of the program itself begins.
- In this case, the directive #include <iostream>, instructs the preprocessor to include a section of standard C++ code, known as *header iostream*, that allows to perform standard input and output operations, such as writing the output of this program (Hello World) to the screen.

std::cout << "Hello World!";

- This line is a C++ statement.
- A statement is an expression that can actually produce some effect.
- Statements are executed in the same order that they appear within a function's body.

Using namespace std

- If you have seen C++ code before, you may have seen cout being used instead of std::cout.
- Both name the same object: the first one uses its unqualified name (cout), while the second qualifies it directly within the namespace std (as std::cout).
- cout is part of the standard library, and all the elements in the standard C++ library are declared within what is called a namespace: the namespace std.

Contd..

- In order to refer to the elements in the std namespace a program shall either qualify each and every use of elements of the library (as we have done by prefixing cout with std::), or introduce visibility of its components. The most typical way to introduce visibility of these components is by means of using declarations:
    - **using namespace std;**

Comments

// This is a C++ program. It prints the sentence:
// Welcome to C++ Programming.

Or

```
/*
   You can include comments that can
   occupy several lines.
*/
```

```
for(j=0; j<n; /* loops n times */ j++)
```

Variable declaration

type variable-name;

Meaning: variable <variable-name> will be a variable of type <type>

Where type can be:

- int //integer
- double //real number
- char //character

Example:

```
int a, b, c;
double x;
int sum;
char my-character;
```

Input statements

cin >> variable-name;

Meaning: read the value of the variable called <variable-name> from the user

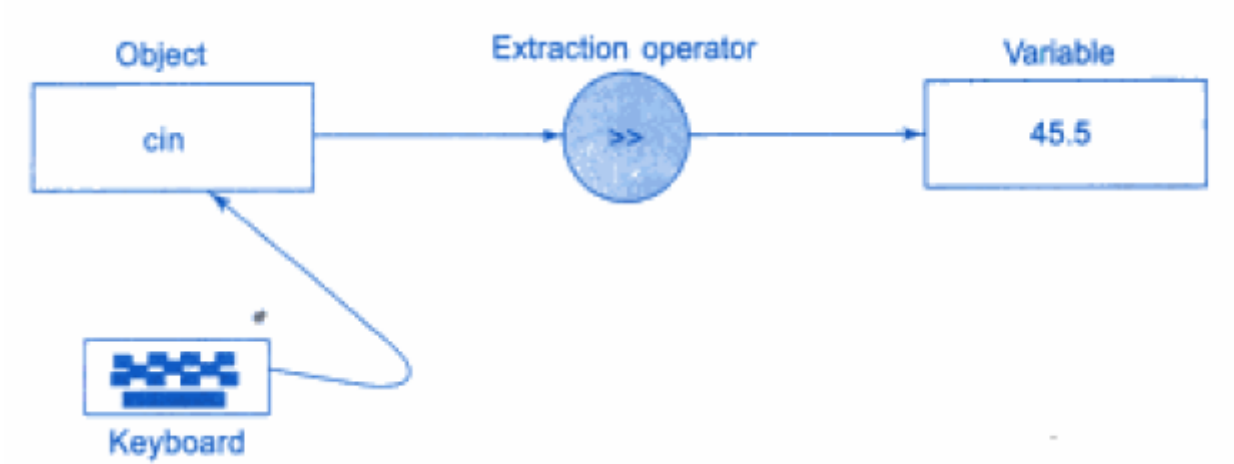
- The operator >> is known as the **extraction or get from** operator.
- It extracts the value from the keyboard and assigns it to the variable on the right.

Example:

```
cin >> a;
cin >> b >> c;
cin >> x;
cin >> my-character;
```

Input Operator

```
cin >> number1;
```



The operator >> is known as *extraction or get from* operator.

Output statements

cout << variable-name;

Meaning: print the value of variable <variable-name> to the user

cout << “any message”;

Meaning: print the message within quotes to the user

cout << endl;

Meaning: print a new line

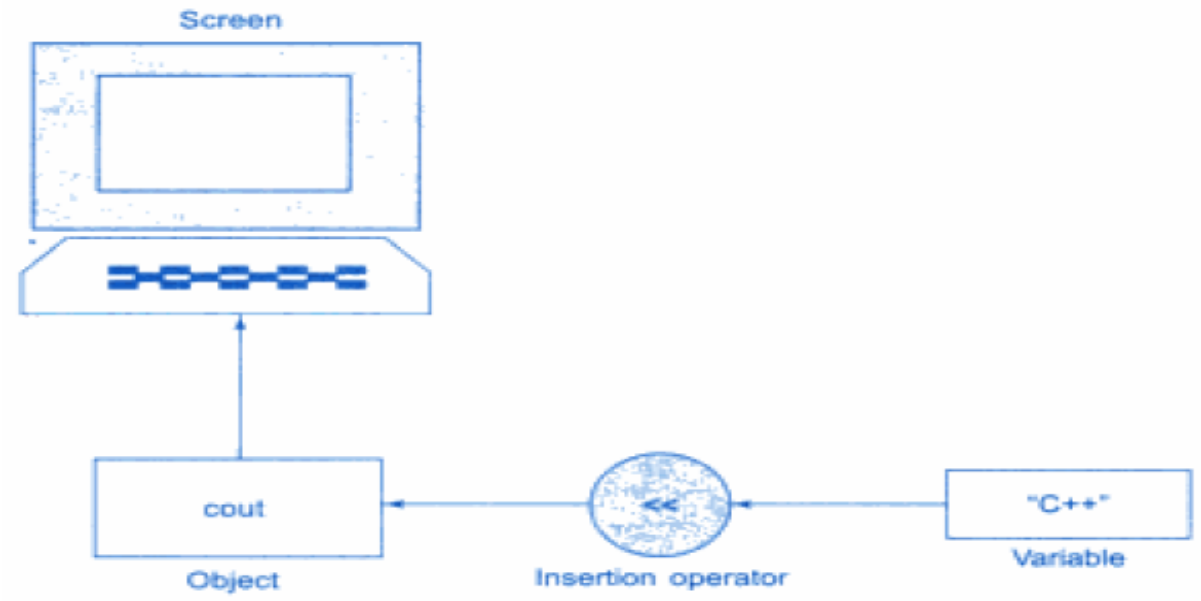
- The operator << is known as the **insertion operator.**
- Multiple use of << in one statement is known as **cascading.**

Example:

```
cout << a;
cout << b << c;
cout << “This is my character: “ << my-character << “ he he he” << endl;
```

Output Operator

cout<< “c++ is better than c”;
cout<<name;



Output using insertion operator

The operator << is called as insertion or put to operator

Cascading I/O Operators

- The multiple use of << or >> in one statement is called cascading.
- Example:-

```
cout<< " sum"<< sum << "\n"
```

- First sends the string "sum=" to cout and then sends the value of sum.
- Finally, it sends the newline character so that the next output will be in the new line.

Example:-

```
cin>>number1>>number2;
```

Access Modifiers

- Accessing a data member depends solely on the access control of that data member.
  This access control is given by Access modifiers in C++.
- There are three access modifiers :
  1. **public**
  2. **private**
  3. **protected**

Contd..

- **Public:** A **public** member is accessible from anywhere outside the class but within a program.
- **Private:** A **private** member variable or function cannot be accessed, or even viewed from outside the class.
- **Protected:** A **protected** member variable or function is very similar to a private member but it provided one additional benefit that they can be accessed in child classes which are called derived classes.